



ANNEXOS - Migració d'una Arquitectura Monolítica a Microserveis

Sergio Santamaría Bayés

Grau en Enginyeria Informàtica

Àrea: GNU/Linux

Tutor: Joaquín López Sánchez-Montañés

Data Lliurament: Gener de 2025

FITXA DEL TREBALL FINAL

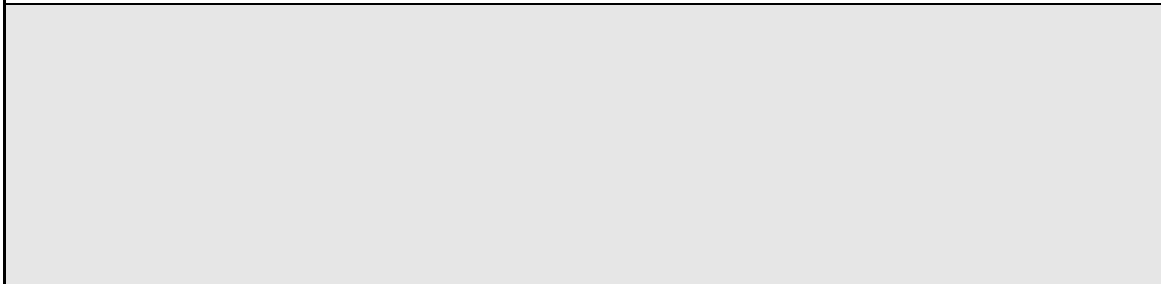
<u>Títol del treball:</u>	<u>Migració d'una Arquitectura Monolítica a Microserveis</u>
Nom de l'autor:	<i>Sergio Santamaría Bayés</i>
Nom del consultor/a:	Joaquin Lopez Sanchez-Montañes
Nom del PRA:	
Data de lliurament (mm/aaaa):	01/2025
Titulació:	Grau en Enginyeria Informàtica
Àrea del Treball Final:	<i>GNU/Linux</i>
Idioma del treball:	<i>Català</i>
Paraules clau	Microservices, Cloud, Arquitectura Monolítica, API, Kubernetes, Docker, HELM Charts, Pipelines CI/CD, DevOps, GitOps
Aquest treball explora la migració d'una aplicació monolítica clàssica cap a una arquitectura basada en microserveis, amb l'objectiu d'analitzar els beneficis i reptes d'aquesta transició. Aprofitant l'experiència adquirida en un projecte real, el context d'aplicació inclou una organització que busca modernitzar la seva infraestructura per millorar la gestió, la flexibilitat i l'eficiència energètica. La metodologia combina pràctiques modernes com <i>DevOps</i> i <i>GitOps</i> per facilitar la transició, amb una anàlisi detallada dels tres components principals de l'aplicació: <i>Front-End</i>, <i>Back-End</i> i base de dades. El projecte es divideix en fases, des del disseny i la planificació fins a la implementació i validació. S'inclouen eines com contenidors, plataformes de <i>Cloud Computing</i> i tècniques d'escalabilitat dinàmica. Els resultats esperats se centren en l'obtenció d'una arquitectura més robusta, flexible i econòmica, amb una optimització significativa dels recursos tecnològics i una reducció del consum energètic. També es preveu una millora en la capacitat de desplegament i manteniment de funcionalitats. Les conclusions apunten a com l'arquitectura de microserveis pot millorar la sostenibilitat tecnològica i ambiental, alhora que es redueixen costos operatius. Aquest treball no només documenta el procés de migració, sinó que també busca demostrar el valor d'aquesta transformació per a empreses que aspiren a mantenir-se competitives en un entorn tecnològic en constant evolució.	

This project explores the migration of a traditional monolithic application to a microservices-based architecture, focusing on analyzing the benefits and challenges of this transformation. Leveraging real-world experience from a current project, the application context involves an organization seeking to modernize its infrastructure to improve management, flexibility, and energy efficiency.

The methodology incorporates modern practices such as DevOps and GitOps to streamline the transition, along with a detailed analysis of the three main application components: Front-End, Back-End, and database. The project is structured into phases, from design and planning to implementation and validation, employing tools like containers, cloud computing platforms, and dynamic scalability techniques.

Expected results include a more robust, flexible, and cost-effective architecture, significantly optimized technological resources, and reduced energy consumption. Additionally, enhanced deployment and maintenance capabilities for application functionalities are anticipated.

The conclusions underline how microservices architecture can enhance technological and environmental sustainability while reducing operational costs. This project not only documents the migration process but also aims to demonstrate the value of such a transformation for businesses striving to remain competitive in an ever-evolving technological landscape.



Índex

1.1. Instal·lació de Laboratori per APP en Arquitectura Monolítica- LAMP.....	5
1.2. Instal·lació de Laboratori per APP en Arquitectura Microserveis.....	17
1.3. Instal·lació de HELM en MiniKube.....	28
1.3.1 Instal·lació d'apps a Kubernetes amb Charts HELM	30
1.5. Instal·lació de Kubernetes Dashboard.....	52
1.6. Migració de MediaWiki de l'entorn monolític a microserveis.....	55

1. Annexos

A continuació es presenten els annexos del laboratori que han abordat diferents aspectes de la instal·lació i configuració d'infraestructures per a l'execució d'aplicacions en entorns monolítics i basats en microserveis.

Tots els experiments s'han realitzat utilitzant un portàtil amb 16GB de RAM i 20 nuclis de processador, amb Debian 12 com a sistema operatiu. Les eines emprades inclouen Visual Studio Code, Git, Vagrant, Virtual Box i Docker Desktop, que han estat essencials per gestionar els entorns de treball i les implementacions de les diverses aplicacions i plataformes com Kubernetes i HELM.

Tot el codi utilitzat es troba allotjat a GitHub en el repositori <https://github.com/sesantama/tfg-monolit2microservices> en el moment d'escriure aquest document.

1.1. Instal·lació de Laboratori per APP en Arquitectura Monolítica- LAMP

Per a poder tenir un laboratori de manera ràpida amb una arquitectura monolítica farem servir Vagrant que permet automatitzar desplegament en entorns de desenvolupament o laboratori en multitud de plataformes. Amb Vagrant es poden desplegar màquines virtuals, imatges de Docker i molt més, tot des de codi de forma similar a com es faria en un entorn productiu de laC.

En el nostre cas, tot i que no sigui del tot habitual en una arquitectura monolítica, farem servir amb Vagrant imatges Docker per a poder desplegar una aplicació de tipus LAMP (Linux Apache MySQL PHP) que donarà servei a una plataforma web, com ara una botiga virtual per e-commerce. En un entorn real tindríem màquines virtuals en corrent en un hipervisor o bé directament màquines físiques *baremetal*.

La instal·lació la farem de manera manual, configurant cada component de manera que simuli com es faria en un entorn monolític on l'administrador ha d'entrar en cada equip i executar les diverses instruccions fins a poder tenir el programari desitjat instal·lat i configurat. S'ha de tenir en compte que no s'ha automatitzat aquest desplegament per tal de ser el més proper a com es faria en un entorn monolític, tot i que al fer servir Docker i Vagrant es podia haver automatitzat tot el procés.

Instal·lació de Docker i Vagrant en Debian 12

Baixarem el paquet debian per docker en la seva versió desktop de <https://docs.docker.com/desktop/setup/install/linux/debian/> i l'instal·larem al nostre equip.

Per poder reproduir-ho farem servir aquest script en bash:

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
# Get Latest Docker Desktop for Debian in https://docs.docker.com/desktop/setup/install/linux/debian/
wget https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb
sudo apt-get install ./docker-desktop-amd64.deb

sergio@sergio-xps-deb:~/Downloads$ sudo apt-get install ./docker-desktop-amd64.deb
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'docker-desktop' instead of './docker-desktop-amd64.deb'
The following additional packages will be installed:
  desktop-file-utils docker-buildx-plugin docker-ce-cli docker-compose-plugin libsuid4 pass qrencode tree uidmap wl-clipboard xclip
Suggested packages:
  libxml-simple-perl python
The following NEW packages will be installed:
  desktop-file-utils docker-buildx-plugin docker-ce-cli docker-compose-plugin docker-desktop libsuid4 pass qrencode tree uidmap wl-clipboard xclip
0 upgraded, 12 newly installed, 0 to remove and 0 not upgraded.
Need to get 58.6 MB/407 MB of archives.
After this operation, 189 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 https://download.docker.com/linux/debian bookworm/stable amd64 docker-ce-cli amd64 5:27.3.1-1-debian.12-bookworm [15.0 MB]
Get:2 https://ftp.debian.org/debian bookworm/main amd64 tree amd64 2.1.0-1 [52.5 kB]
Get:3 https://ftp.debian.org/debian bookworm/main amd64 pass all 1.7.4-6 [34.0 kB]
Get:4 https://ftp.debian.org/debian bookworm/main amd64 desktop-file-utils amd64 0.26-1 [91.2 kB]
Get:5 https://ftp.debian.org/debian bookworm/main amd64 libsuid4 amd64 1:4.13+dfsg1-1+b1 [211 kB]
Get:6 https://ftp.debian.org/debian bookworm/main amd64 uidmap amd64 1:4.13+dfsg1-1+b1 [189 kB]
Get:7 https://ftp.debian.org/debian bookworm/main amd64 qrencode amd64 4.1.1-1 [42.6 kB]
Get:8 https://ftp.debian.org/debian bookworm/main amd64 wl-clipboard amd64 2.1.0-0.1+b1 [25.9 kB]
Get:9 https://ftp.debian.org/debian bookworm/main amd64 xclip amd64 0.13-2 [23.3 kB]
Get:10 /home/sergio/Downloads/docker-desktop-amd64.deb docker-desktop amd64 4.35.1-173168 [349 MB]
Get:11 https://download.docker.com/linux/debian bookworm/stable amd64 docker-buildx-plugin amd64 0.17.1-1-debian.12-bookworm [30.3 MB]
Get:12 https://download.docker.com/linux/debian bookworm/stable amd64 docker-compose-plugin amd64 2.29.7-1-debian.12-bookworm [12.7 MB]
Fetched 58.6 MB in 15s (3.982 kB/s)
Selecting previously unselected package docker-ce-cli.
(Reading database ... 259524 files and directories currently installed.)
```

```
[sudo] password for sergio:
Hit:1 https://security.debian.org/debian-security bookworm-security InRelease
Hit:2 https://ftp.debian.org/debian bookworm InRelease
Hit:3 https://ftp.debian.org/debian bookworm-updates InRelease
Hit:4 https://download.virtualbox.org/virtualbox/debian bookworm InRelease
Hit:5 https://packages.microsoft.com/repos/code stable InRelease
Hit:6 https://download.docker.com/linux/debian bookworm InRelease
Hit:7 https://ftp.debian.org/debian bookworm-proposed-updates InRelease
Hit:8 https://ftp.debian.org/debian bookworm-backports InRelease
Get:9 https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease [1,596 B]
Hit:10 https://dl.google.com/linux/chrome/deb stable InRelease
Err:9 https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
Fetched 1,596 B in 1s (1,176 B/s)
Reading package lists... Done
W: An error occurred during the signature verification. The repository is not updated and the previous index files will be used. GPG error: https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
W: Failed to fetch https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./InRelease The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
W: Some index files failed to download. They have been ignored, or old ones used instead.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20230311).
curl is already the newest version (7.88.1-10+deb12u8).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Hit:1 https://security.debian.org/debian-security bookworm-security InRelease
Hit:2 https://download.virtualbox.org/virtualbox/debian bookworm InRelease
Hit:3 https://ftp.debian.org/debian bookworm InRelease
Hit:4 https://download.docker.com/linux/debian bookworm InRelease
Hit:5 https://packages.microsoft.com/repos/code stable InRelease
Hit:6 https://ftp.debian.org/debian bookworm-updates InRelease
Hit:7 https://ftp.debian.org/debian bookworm-proposed-updates InRelease
Get:8 https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease [1,596 B]
Hit:9 https://ftp.debian.org/debian bookworm-backports InRelease
Hit:10 https://dl.google.com/linux/chrome/deb stable InRelease
Err:9 https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease
  The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
Fetched 1,596 B in 1s (1,370 B/s)
Reading package lists... Done
W: An error occurred during the signature verification. The repository is not updated and the previous index files will be used. GPG error: https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./ InRelease: The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
W: Failed to fetch https://download.opensuse.org/repositories/home:/npreining:/debian-ubuntu-onedrive/Debian_12_./InRelease The following signatures couldn't be verified because the public key is not available: NO_PUBKEY B8AC3986876D887E
W: Some index files failed to download. They have been ignored, or old ones used instead.
--2023-11-23 20:53:01-- https://desktop.docker.com/linux/main/amd64/docker-desktop-amd64.deb
Resolving desktop.docker.com (desktop.docker.com)... 52.84.98.115, 52.84.98.7, 52.84.98.69, ...
Connecting to desktop.docker.com (desktop.docker.com)|52.84.98.115|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 348254088 (333M)
Saving to: 'docker-desktop-amd64.deb'

docker-desktop-amd64.deb 21%[=====] 70.80M 4.89MB/s eta 47s
```

Verifiquem que quedin tots els paquets instal·lats:

```
sergio@sergio-xps-deb:~/OneDrive/TF6/Project$ sudo dkg -l | grep docker
ll docker 1:5-2 all transitional package
ll docker-buildx-plugin 0.17.1-1-debian.12-bookworm amd64 Docker Buildx cli plugin.
ll docker-ce-cli 5:27.3.1-1-debian.12-bookworm amd64 Docker CLI: the open-source application container engine
ll docker-compose-plugin 2.29.7-1-debian.12-bookworm amd64 Docker Compose (v2) plugin for the Docker CLI.
ll docker-desktop 4.35.1-173168 amd64 Docker Desktop is an easy-to-install application that enables you to locally build and share containerized applications and microservices. It includes Docker Engine, Docker CLI client, Docker Compose, Docker Content Trust, Kubernetes, and Credential Helper. Docker Desktop runs a light-weight Linux VM to provide an isolated local container runtime, and an experience consistent with Mac and Windows versions of Docker Desktop.
ll docker 1:5-2 amd64 System tray for KDE3/GNOME2 docklet applications
```

TFG – Plataforma GNU/Linux

Annexos

Migració d'una Arquitectura Monolítica a Microserveis

Seguidament instal·larem Vagrant amb les biblioteques de `libvirt` per a poder executar VMs i contenidors Docker.

```
sergio@sergio-xps-deb:~$ sudo apt install vagrant-libvirt libvirt-daemon-system vagrant
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gnutls-bin iptables libip6tc2 libnss-mymachines libtpms0 libvirt-clients libvirt-daemon libvirt-daemon-config-network
  libvirt-daemon-config-nwfilter libvirt-daemon-driver-lxc libvirt-daemon-driver-qemu libvirt-daemon-driver-vbox
  libvirt-daemon-driver-xen libvirt-daemon-system libvirt-daemon-system-systemd libxencall1 libxendevicemodel1 libxenevtchn1 libxenforeignmemory1
  libxengnttab1 libxenhypfs1 libxenmisc4.17 libxenstore4 libxentoolcore1 libxentoollog1 mdevctl netcat-openbsd swtpm swtpm-lib
  swtpm-tools systemd-container
Suggested packages:
  firewall libvirt-clients-qemu libvirt-login-shell libvirt-daemon-driver-storage-gluster
  libvirt-daemon-driver-storage-iscsi-direct libvirt-daemon-driver-storage-rbd libvirt-daemon-driver-storage-zfs numad auditd
  open-iscsi pm-utils systemtap zfsutils trousers virtualbox
The following NEW packages will be installed:
  gnutls-bin iptables libip6tc2 libnss-mymachines libtpms0 libvirt-clients libvirt-daemon libvirt-daemon-config-network
  libvirt-daemon-config-nwfilter libvirt-daemon-driver-lxc libvirt-daemon-driver-qemu libvirt-daemon-driver-vbox
  libvirt-daemon-driver-xen libvirt-daemon-system libvirt-daemon-system-systemd libxencall1 libxendevicemodel1 libxenevtchn1
  libxenforeignmemory1 libxengnttab1 libxenhypfs1 libxenmisc4.17 libxenstore4 libxentoolcore1 libxentoollog1 mdevctl netcat-openbsd
  swtpm swtpm-lib swtpm-tools systemd-container vagrant vagrant-libvirt
0 upgraded, 33 newly installed, 0 to remove and 43 not upgraded.
Need to get 6,867 kB of archives.
After this operation, 24.0 MB of additional disk space will be used.
Do you want to continue? [Y/n]
```

Verifiquem que quedi ben instal·lat:

```
sergio@sergio-xps-deb:~$ sudo dpkg -l | grep vagrant
ii  ruby-vagrant-cloud      3.0.5-1          all          Vagrant Cloud API Library
ii  vagrant                 2.3.4+dfsg-1     all          Tool for building and distributing virtualized development environments
ii  vagrant-libvirt         0.11.2-1         all          Vagrant plugin that adds an Libvirt provider to Vagrant
sergio@sergio-xps-deb:~$
```

Configuració de la imatge Docker

Comencem creant una imatge de Docker que s'assembla molt a un entorn Vagrant. El `Dockerfile` que utilitzarem conté tots els components necessaris, inclòs SSH, `systemd`. Aquesta imatge servirà de base per a les nostres màquines virtuals.

```
# Docker image to use with Vagrant
# Aims to be as similar to normal Vagrant usage as possible
# Use the official Ubuntu base image
FROM ubuntu:22.04
# Set the environment variable to indicate it's a Docker container
ENV container=docker
# First, update the package repository and perform a system upgrade
RUN apt-get update && apt-get dist-upgrade -y
# Install necessary packages including SSH, sudo, libffi-dev, systemd, and openssh-client
RUN apt-get install -y --no-install-recommends ssh sudo libffi-dev systemd openssh-client
# The following instructions were in a separate block, so they are now correctly merged.
# Install the SSH server
RUN apt-get install -y openssh-server wget
# Create the Vagrant user and set up SSH access
RUN useradd -m -s /bin/bash vagrant && \
  echo 'vagrant:vagrant' | chpasswd && \
  mkdir -p /home/vagrant/.ssh && \
  chmod 700 /home/vagrant/.ssh && \
  wget --no-check-certificate \
  https://raw.githubusercontent.com/hashicorp/vagrant/main/keys/vagrant.pub \
  -O /home/vagrant/.ssh/authorized_keys && \
  chmod 600 /home/vagrant/.ssh/authorized_keys && \
  chown -R vagrant:vagrant /home/vagrant/.ssh && \
  echo '%vagrant ALL=NOPASSWD:ALL' > /etc/sudoers.d/vagrant
# Enable password-based authentication for SSH
RUN sed -i 's/PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config
# Enable SSH key-based authentication
RUN echo y | ssh-keygen -t rsa -N "" -f /etc/ssh/ssh_host_rsa_key
RUN echo y | ssh-keygen -t ecdsa -N "" -f /etc/ssh/ssh_host_ecdsa_key
RUN echo y | ssh-keygen -t ed25519 -N "" -f /etc/ssh/ssh_host_ed25519_key
RUN mkdir -p /run/ssh
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D", "-E", "/var/log/ssh.log"]
```

D'aquesta manera ja podem crear la nostra imatge base de sistema operatiu amb Docker per a Vagrant, basada en la Ubuntu 22.04, fent servir la comanda `docker build -t vagrant_vm:latest .`

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$ docker build -t vagrant_vm:latest .
[+] Building 1.2s (14/14) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.74kB
=> [internal] load metadata for docker.io/library/ubuntu:22.04
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [ 1/10] FROM docker.io/library/ubuntu:22.04@sha256:0e5e4a57c2499249aafc3b40fcd541e9a456aab7296681a3994d631587203f97
=> CACHED [ 2/10] RUN apt-get update && apt-get dist-upgrade -y
=> CACHED [ 3/10] RUN apt-get install -y --no-install-recommends ssh sudo libffi-dev systemd openssh-client
=> CACHED [ 4/10] RUN apt-get install -y openssh-server wget
=> CACHED [ 5/10] RUN useradd -m -s /bin/bash vagrant && echo 'vagrant:vagrant' | chpasswd && mkdir -p /home/vagrant/.ssh && chmod 700 /home/vagrant/.ss
=> CACHED [ 6/10] RUN sed -i 's/PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config
=> CACHED [ 7/10] RUN echo y | ssh-keygen -t rsa -N "" -f /etc/ssh/ssh_host_rsa_key
=> CACHED [ 8/10] RUN echo y | ssh-keygen -t ecdsa -N "" -f /etc/ssh/ssh_host_ecdsa_key
=> CACHED [ 9/10] RUN echo y | ssh-keygen -t ed25519 -N "" -f /etc/ssh/ssh_host_ed25519_key
=> CACHED [10/10] RUN mkdir -p /run/ssh
=> exporting to image
=> exporting layers
=> writing image sha256:2693a56d578971d180382906e99dc17452d823f37632da67002df02aa5c80e29
=> naming to docker.io/library/vagrant_vm:latest
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ctjjty0cebkgdysy881tmhcji
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
vagrant_vm	latest	2693a56d5789	4 minutes ago	164MB

Ara que la nostra imatge de Docker està preparada, configurem Vagrant perquè l'utilitzi. El nostre `Vagrantfile` defineix un node master on instal·larem tots els components, noteu que redirigim el port 80 cap al 8080 del nostre pc per poder accedir-hi.

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|
  config.vm.provider :docker do |docker, override|
    override.vm.box = nil
    docker.image = "vagrant_vm:latest"
    docker.remains_running = true
    docker.has_ssh = true
    docker.privileged = true
    docker.volumes = ["/sys/fs/cgroup:/sys/fs/cgroup:rw"]
    docker.create_args = ["--cgroupns=host"]
    # Uncomment to force arm64 for testing images
    # docker.create_args = ['--platform=linux/arm64']
  end

  # Master Node Configuration
  config.vm.define "master" do |master|
    master.vm.hostname = "master"
    master.vm.network "private_network", type: "dhcp"
    master.vm.network "forwarded_port", guest: 80, host: 8080
  end
end
```

Arrancarem la màquina amb `vagrant up`:

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$ vagrant up
Bringing machine 'master' up with 'docker' provider...
==> master: Creating and configuring docker networks...
==> master: Enabling network interfaces...
==> master: Starting container...
==> master: Waiting for machine to boot. This may take a few minutes...
    master: SSH address: 127.0.0.1:2222
    master: SSH username: vagrant
    master: SSH auth method: private key
==> master: Machine booted and ready!
==> master: Machine already provisioned. Run `vagrant provision` or use the `--provision`
==> master: flag to force provisioning. Provisioners marked to run always will still run.
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$
```

Desplegament de LAMP

La part principal del projecte és desplegar un LAMP. Això inclou (Linux) Apache, MySQL i PHP. Ens assegurem que Apache s'executa i està configurat per iniciar-se durant l'arrencada, assegurem la instal·lació de MySQL i validem la funcionalitat de PHP amb Apache.

```
vagrant ssh master -c "sudo apt update && sudo apt install -y apache2 mysql-
server php libapache2-mod-php"
```

Un cop instal·lat els arrencarem:

```
#Configurar Apache i MySQL als dos nodes

vagrant ssh master -c "sudo service apache2 start"
vagrant ssh master -c "sudo service mysql start"
vagrant ssh master -c "sudo mysql_secure_installation"

# Test configuració de LAMP
vagrant ssh master -c "echo '<?php phpinfo(); ?>' | sudo tee
/var/www/html/info.php"
```

```

Securing the MySQL server deployment.

Connecting to MySQL using a blank password.

VALIDATE PASSWORD COMPONENT can be used to test passwords
and improve security. It checks the strength of password
and allows the users to set only those passwords which are
secure enough. Would you like to setup VALIDATE PASSWORD component?

Press y|Y for Yes, any other key for No: y

There are three levels of password validation policy:

LOW      Length >= 8
MEDIUM  Length >= 8, numeric, mixed case, and special characters
STRONG Length >= 8, numeric, mixed case, special characters and dictionary      file

Please enter 0 = LOW, 1 = MEDIUM and 2 = STRONG: 0

Skipping password set for root as authentication with auth_socket is used by default.
If you would like to use password authentication instead, this can be done with the "ALTER_USER" command.
See https://dev.mysql.com/doc/refman/8.0/en/alter-user.html#alter-user-password-management for more information.

By default, a MySQL installation has an anonymous user,
allowing anyone to log into MySQL without having to have
a user account created for them. This is intended only for
testing, and to make the installation go a bit smoother.
You should remove them before moving into a production
environment.

Remove anonymous users? (Press y|Y for Yes, any other key for No) : y
Success.

Normally, root should only be allowed to connect from
'localhost'. This ensures that someone cannot guess at
the root password from the network.

Disallow root login remotely? (Press y|Y for Yes, any other key for No) : n

... skipping.
By default, MySQL comes with a database named 'test' that
anyone can access. This is also intended only for testing,
and should be removed before moving into a production
environment.

Remove test database and access to it? (Press y|Y for Yes, any other key for No) : y
- Dropping test database...
Success.

- Removing privileges on test database...
Success.

Reloading the privilege tables will ensure that all changes
made so far will take effect immediately.

Reload privilege tables now? (Press y|Y for Yes, any other key for No) : y
Success.

All done!
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$

```

Si esta tot ok, reiniciem apache per a que agafi els canvis

```

sergio@sergio-xps-deb:~/OneDrive/TFG/Project$ vagrant ssh master -c "sudo service apache2 restart"
* Restarting Apache httpd web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.2. Set the 'ServerName' directive globally to suppress this message

```

Si fem un curl a localhost desde master ens mostrarà el contingut del apache.

```
vagrant@master:~$ curl http://localhost
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
      * {
        margin: 0px 0px 0px 0px;
        padding: 0px 0px 0px 0px;
      }

      body, html {
        padding: 3px 3px 3px 3px;

        background-color: #D8DBE2;

        font-family: Ubuntu, Verdana, sans-serif;
        font-size: 11pt;
        text-align: center;
      }

      div.main_page {
        position: relative;
        display: table;
      }
    </style>
  </head>
  <body>
    <div class="main_page">
      <div class="header">
        <img alt="Ubuntu logo" data-bbox="355 638 385 668"/>
        <h1>Apache2 Default Page</h1>
        <h2>Ubuntu</h2>
        <div class="button">It works!</div>
      </div>
      <div class="text">
        <p>This is the default welcome page used to test the correct operation of the Apache2 server after installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should replace this file (located at /var/www/html/index.html) before continuing to operate your HTTP server.
        <p>If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.
      </div>
      <div class="configuration">
        <h3>Configuration Overview</h3>
        <p>Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is fully documented in /usr/share/doc/apache2/README.Debian.gz. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the manual if the apache2-doc package was installed on this server.
        <p>The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:


```
/etc/apache2/
|-- apache2.conf
| |-- ports.conf
|-- mods-enabled
| |-- *.load
| |-- *.conf
|-- conf-enabled
| |-- *.conf
|-- sites-enabled
| |-- *.conf
```


        <ul>
          <li>• apache2.conf is the main configuration file. It puts the pieces together by including all remaining configuration files when starting up the web server.
          <li>• ports.conf is always included from the main configuration file. It is used to determine the listening ports for incoming connections and this file can be customized online.
        </ul>
      </div>
    </div>
  </body>
</html>
```

Per poder veure el contingut des del navegador del nostre pc, farem un tunel SSH:
ssh -L 8080:localhost:80 -i master_public_key -p 2222 vagrant@127.0.0.1

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project$ ssh -L 8080:localhost:80 -i master_public_key -p 2222 vagrant@127.0.0.1
vagrant@127.0.0.1's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.10.11-linuxkit x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Fri Dec 27 17:44:43 2024 from 172.17.0.1
vagrant@master:~$
```



Instal·lació de l'aplicació MediaWiki

Per a poder provar el nostre *stack* LAMP necessitem una aplicació corrent, per això instal·larem un servidor de wiki amb MediaWiki (<https://www.mediawiki.org/wiki/MediaWiki/es>) el motor que fa servir la Wikipedia i milers de llocs webs allotjant documents a Internet.

Per instal·lar MediaWiki en Ubuntu farem primerament la instal·lació dels paquets necessaris: `sudo apt install mediawiki`

```
vagrant@master:~$ sudo apt install mediawiki
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libdatatriel liblua5.1-0 libonig5 libpython3-stdlib libthai-data libthai0 libyaml-0-2 mediawiki-classes php-apcu php-curl php-gd php-gmp php-intl php-luasandbox
  php-mbstring php-mysql php-wikidiff2 php-xml php-yaml php8.1-apcu php8.1-curl php8.1-gd php8.1-gmp php8.1-intl php8.1-mbstring php8.1-mysql php8.1-xml php8.1-yaml python3
  python3-minimal python3.10 python3.10-minimal
Suggested packages:
  memcached clamav firejail python3-doc python3-tk python3-venv python3.10-doc binutils binfmt-support
The following NEW packages will be installed:
  libdatatriel liblua5.1-0 libonig5 libpython3-stdlib libthai-data libthai0 libyaml-0-2 mediawiki mediawiki-classes php-apcu php-curl php-gd php-gmp php-intl php-luasandbox
  php-mbstring php-mysql php-wikidiff2 php-xml php-yaml php8.1-apcu php8.1-curl php8.1-gd php8.1-gmp php8.1-intl php8.1-mbstring php8.1-mysql php8.1-xml php8.1-yaml python3
  python3-minimal python3.10 python3.10-minimal
0 upgraded, 33 newly installed, 0 to remove and 0 not upgraded.
Need to get 36.2 MB of archives.
After this operation, 206 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3.10-minimal amd64 3.10.12-1-22.04.7 [2279 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-minimal amd64 3.10.6-1-22.04.1 [24.3 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3.10 amd64 3.10.12-1-22.04.7 [509 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpython3-stdlib amd64 3.10.6-1-22.04.1 [6812 B]
[...]
debconf: falling back to frontend: Readline

Creating config file /etc/php/8.1/mods-available/mbstring.ini with new version
Setting up php-luasandbox (4.0.2-3build1) ...
Setting up php-yaml (2.2.2+2.1.0+2.0.4+1.3.2-5) ...
Setting up python3 (3.10.6-1-22.04.1) ...
running python rtupdate hooks for python3.10...
running python post-rtupdate hooks for python3.10...
Setting up php-mbstring (2:8.1+92ubuntu1) ...
Setting up php-intl (2:8.1+92ubuntu1) ...
Setting up mediawiki (1:1.35.6-1) ...
apache2_invoke: Enable configuration mediawiki
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of reload.
Setting up libthai0:amd64 (0.1.29-1build1) ...
Setting up php-apcu (5.1.21+4.0.11-7ubuntu1) ...
Setting up php-wikidiff2 (1.13.0-1build1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.8) ...
Processing triggers for libapache2-mod-php8.1 (8.1.2-1ubuntu2.20) ...
Processing triggers for php8.1-cgi (8.1.2-1ubuntu2.20) ...
vagrant@master:~$
```

Un cop instal·lat el paquet de mediawiki, hem de configurar l'apache per servir els fitxers de la MediaWiki, per això configurem un host virtual a l'apache per a MediaWiki:

- Creem un fitxer anomenat `/etc/apache2/sites-available/mediawiki.conf` amb el següent contingut:

```
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/lib/mediawiki
    <Directory /var/lib/mediawiki>
        Options +FollowSymLinks
        AllowOverride All
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/mediawiki-error.log
    CustomLog ${APACHE_LOG_DIR}/mediawiki-access.log combined
</VirtualHost>
```

- Ara habilitem el nou host virtual executant el següent:

```
sudo a2ensite mediawiki.conf
sudo service apache2 reload
```

```
vagrant@master:~$ sudo a2ensite mediawiki.conf
sudo service apache2 reload
Enabling site mediawiki.
To activate the new configuration, you need to run:
    service apache2 reload
* Reloading Apache httpd web server apache2
*
vagrant@master:~$
```

Un cop ja tenim el servidor web hem de configurar la base de dades i l'usuari de MySQL per la MediaWiki.

- Primer, iniciarem sessió en el servidor MySQL amb el usuari root:
`mysql -u root -p`
- Tot seguit, creem una nova base de dades i un usuario per MediaWiki amb les sentències SQL següents:

```
CREATE DATABASE wikidb;
CREATE USER 'wikiuser'@'localhost' IDENTIFIED BY 'your_password';
GRANT ALL PRIVILEGES ON wikidb.* TO 'wikiuser'@'localhost';
FLUSH PRIVILEGES;
```

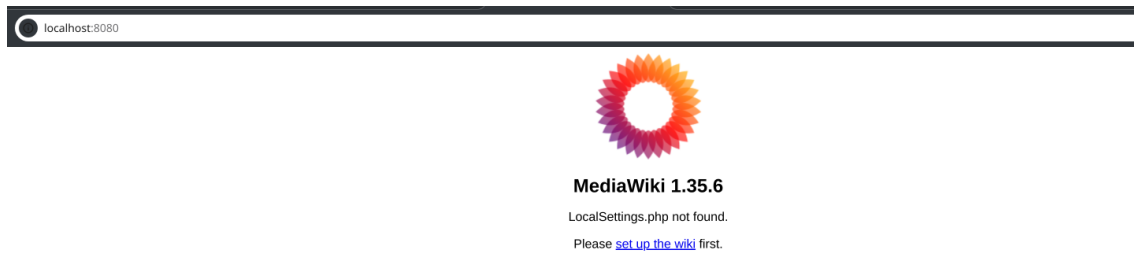
Reemplacem 'password' per una contrasenya segura. Això ens crearà una nova DB *wikidb* i un nou usuari amb privilegis sobre aquesta DB anomenat *wikiuser* amb la contrasenya especificada.

Comprovem amb `SHOW GRANTS FOR 'wikiuser'@'localhost';`

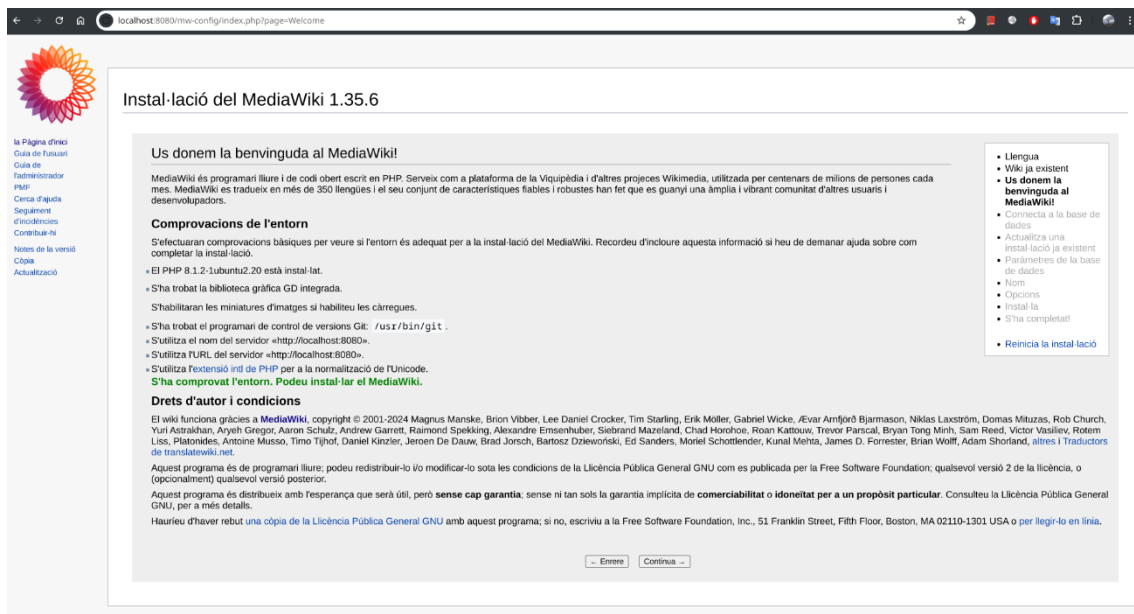
```
mysql> SHOW GRANTS FOR 'wikiuser'@'localhost';
+-----+
| Grants for wikiuser@localhost |
+-----+
| GRANT USAGE ON *.* TO `wikiuser`@`localhost` |
| GRANT ALL PRIVILEGES ON `wikidb`.* TO `wikiuser`@`localhost` |
+-----+
2 rows in set (0.00 sec)

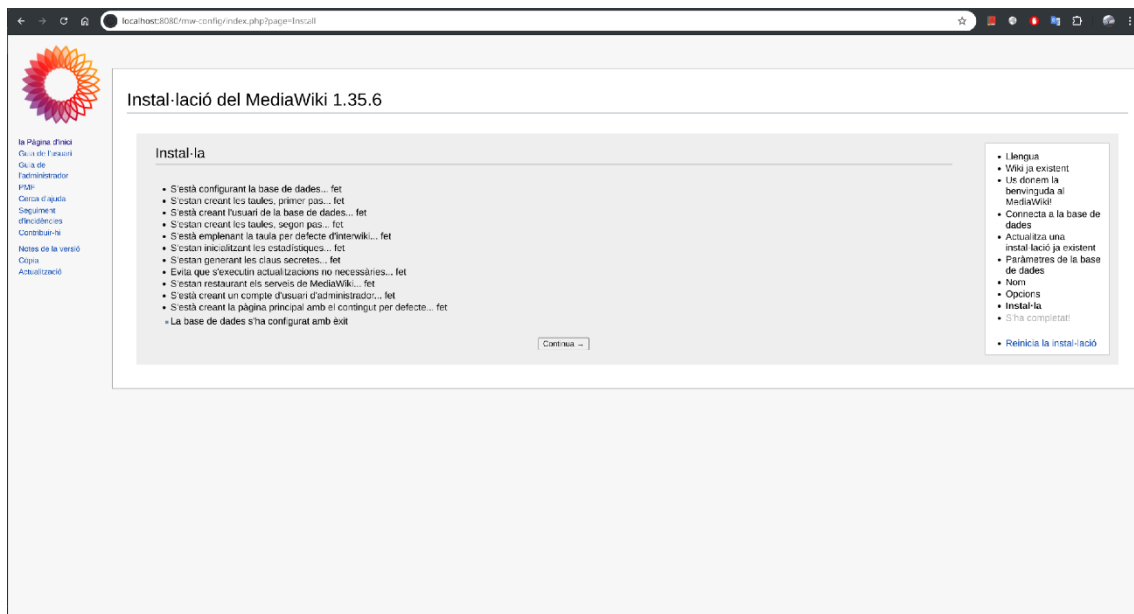
mysql>
```

En obrir la pàgina al navegador ja esta disponible MediaWiki:

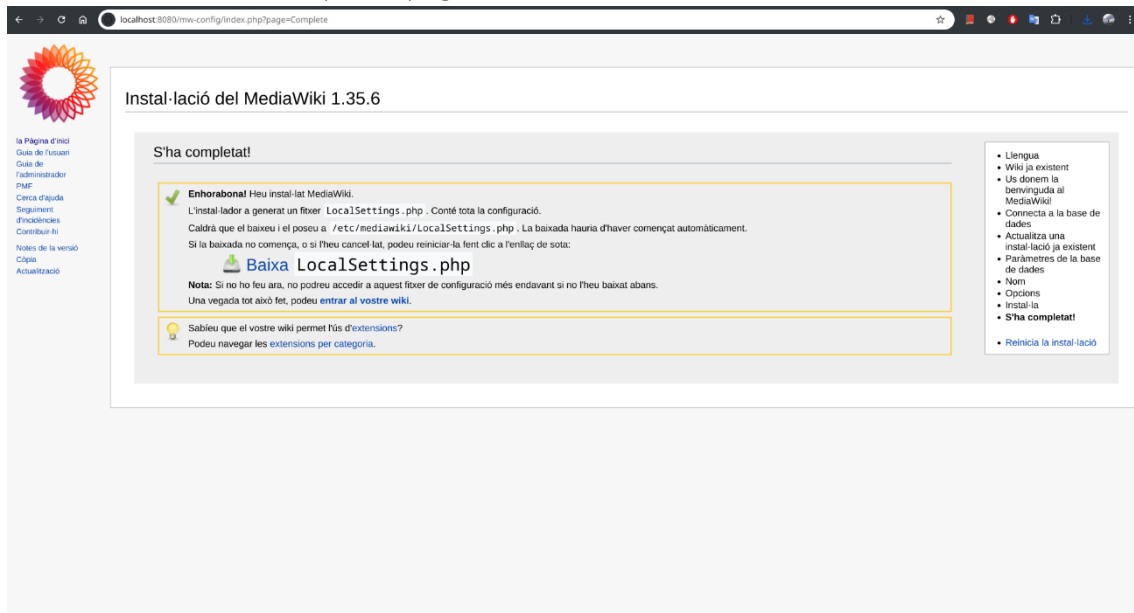


Procedirem a la instal·lació seguint els passos en pantalla.

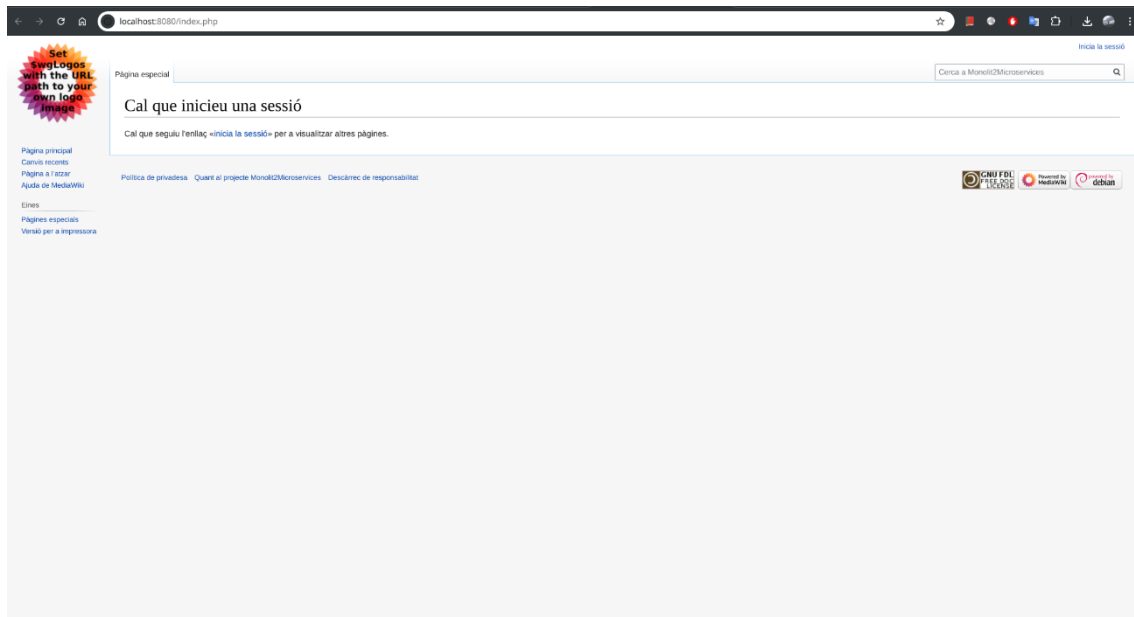




Finalment ha de sortir aquesta pàgina:



Ens baixem el fitxer `LocalSettings.php` i el copiem a `/etc/mediawiki`



Ja tenim la nostra aplicació monolítica desplegada i ja podem realitzar la migració a microserveis.

1.2. Instal·lació de Laboratori per APP en Arquitectura Microserveis

Hi ha diverses maneres de tenir un clúster de Kubernetes en funcionament, mencionarem algunes opcions perquè els lectors en tinguin coneixement tot i que no les provarem totes.

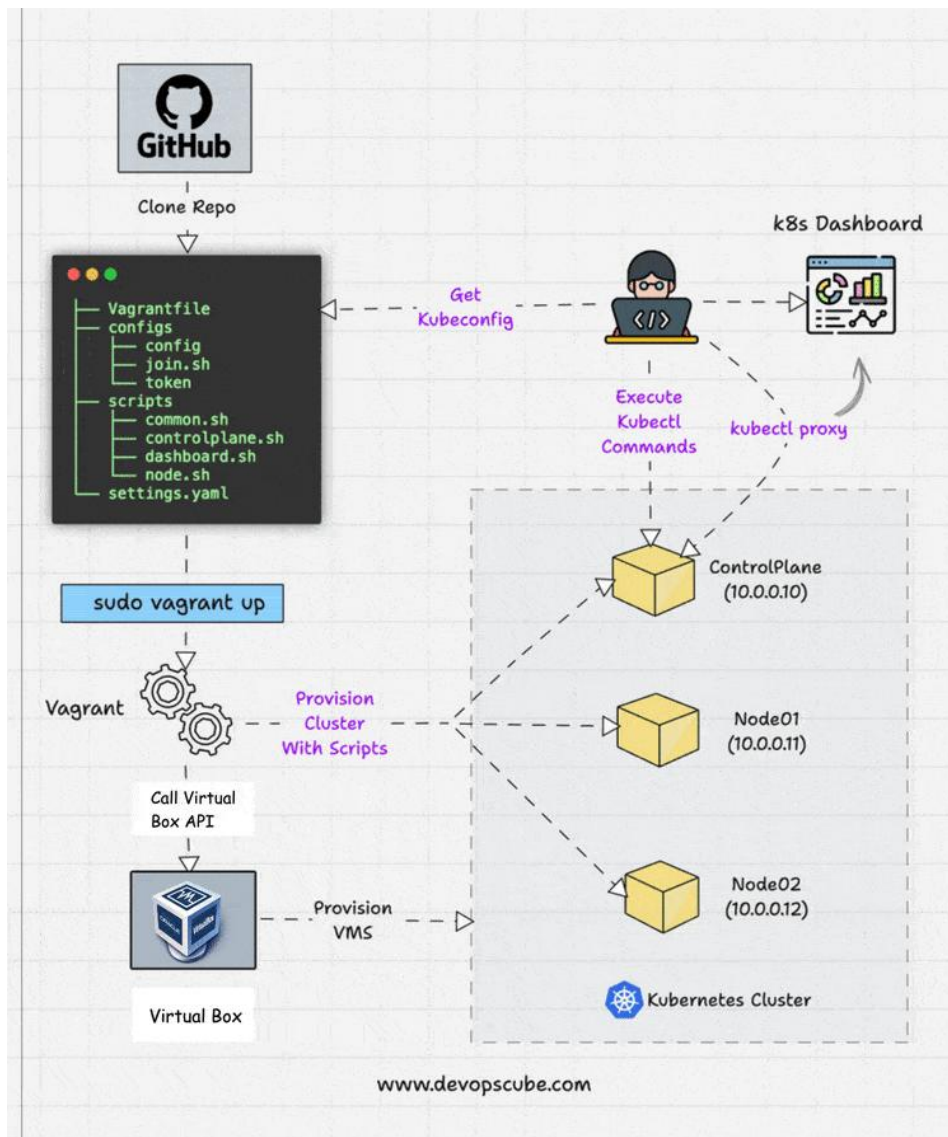
En primer lloc, tenim les instal·lacions de Kubernetes en entorns locals, on destaquen eines com Minikube i Docker Desktop. Els laboratoris d'aquest projecte relacionats amb arquitectura de microserveis es realitzaran amb Minikube, tot i que, en principi, podríem utilitzar qualsevol altra distribució.

Si volem llançar clústers de Kubernetes al núvol, una opció interessant és Kops, un projecte de codi obert que permet desplegar clústers de Kubernetes en múltiples proveïdors de serveis al núvol. És una eina molt ben dissenyada, però amb l'aparició de solucions pròpies dels proveïdors de núvol, el seu ús pot estar disminuint.

Avui en dia, si necessites desplegar un clúster de Kubernetes al núvol, és recomanable utilitzar les solucions natives de cada proveïdor, com ara AKS (Azure Kubernetes Service), EKS (Elastic Kubernetes Service d'AWS), GKE (Google Kubernetes Engine) o el servei de Kubernetes de Digital Ocean. Si aquestes solucions no són prou flexibles per als casos d'ús específics, Kops podria ser una opció a considerar.

Per a poder desplegar una aplicació en arquitectura de microserveis, com ara la MediaWiki que hem desplegat en entorn monolític, farem servir doncs MiniKube per tenir el nombre cluster de Kubernetes. Les VMs que tindren aquest programari estaren desplegades com hem fet abans amb Vagrant per poder obtenir màquines de manera ràpida i molt semblant a com es desplegarien en un on-premises o al núvol. Els contenidors que orquestrarà el nostre MiniKube seràn en Docker.

Per a tenir un clúster funcional, desplegarem 3 VMs fent servir Vagrant i en aquesta ocasió sobre VirtualBox adaptant la guia de [DevopsCube](#)



<https://devopscube.com/kubernetes-cluster-vagrant/>

- **master** (ControlPlane al diagrama): serà el node mestre de Kubernetes on desplegar el Control Plane i la resta d'eines per controlar Kubernetes
- **worker1** (Node01 al diagrama): Node 1 on correran els nostres Pods de kubernetes amb els contenidors Docker dins
- **worker2** (Node02 al diagrama): Node 2 on correran els nostres Pods de kubernetes amb els contenidors Docker dins

A continuació el fitxer VagrantFile per desplegar el laboratori amb `vagrant up`

```
Vagrant.configure("2") do |config|
  config.vm.provider :virtualbox do |v|
    v.memory = 2048
    v.cpus = 1
  end

  config.vm.define :master do |master|
    config.vm.boot_timeout = 600
    config.vm.box_download_insecure = true
    #config.ssh.insert_key = false
    master.vm.box = "learnk8s/networking"
    master.vm.box_version = ".0.0"
    master.vm.hostname = "master"
    master.vm.network :private_network, ip: "10.0.0.10"
    master.vm.provision :shell, privileged: false, inline: $provision_mas-
ter
    master.vm.provider :virtualbox do |vb|
      vb.cpus = 2
      vb.name = "master"
    end
  end

  %w{worker1 worker2}.each_with_index do |name, i|
    config.vm.define name do |worker|
      config.vm.boot_timeout = 600
      config.vm.box_download_insecure = true
      #config.ssh.insert_key = false
      worker.vm.box = "learnk8s/networking"
      worker.vm.box_version = "3.0.0"
      worker.vm.hostname = name
      worker.vm.provider :virtualbox do |vb|
        vb.customize ["modifyvm", :id, "--memory", "2048"]
      end
      vb.name = name
      worker.vm.network :private_network, ip: "10.0.0.#{i + 11}"
      worker.vm.provision :shell, privileged: false, inline: <<-SHELL
sudo /vagrant/join.sh
echo 'Environment="KUBELET_EXTRA_ARGS=--node-ip=10.0.0.#{i + 11}"' | \
  sudo tee -a /etc/systemd/system/kubelet.service.d/10-kubeadm.conf
sudo systemctl daemon-reload
sudo systemctl restart kubelet
sudo systemctl enable docker
SHELL
    end
  end
end
```

```

end
end

$provision_master = <<-SHELL
OUTPUT_FILE=/vagrant/join.sh
rm -rf $OUTPUT_FILE

# Start cluster
sudo kubeadm init \
  --ignore-preflight-errors=SystemVerification \
  --apiserver-advertise-address=10.0.0.10 \
  --pod-network-cidr=10.244.0.0/16 \
  --kubernetes-version v1.20.15 | \
  grep -A1 "kubeadm join" > ${OUTPUT_FILE}
chmod +x $OUTPUT_FILE

# Configure kubectl
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
cp $HOME/.kube/config /vagrant
cp /kube/nginx-ingress-v0.44.0.yaml /vagrant
sudo apt-get install bash-completion
echo 'source <(kubectl completion bash)' >> ~/.bashrc
# Fix kubelet IP
echo 'Environment="KUBELET_EXTRA_ARGS=--node-ip=10.0.0.10"' | \
  sudo tee -a /etc/systemd/system/kubelet.service.d/10-kubeadm.conf

# Configure flannel
kubectl apply -f /kube/kube-flannel-v0.13.0.yaml

sudo systemctl daemon-reload
sudo systemctl restart kubelet
sudo systemctl enable docker

SHELL

```

El fitxer comença amb `Vagrant.configure("2")`, que defineix que s'utilitza la versió 2 de l'API de Vagrant. Configura les màquines virtuals (VM) amb VirtualBox, assignant-los 2 GB de memòria RAM i 1 CPU per defecte.

Configuració del Node Màster

- Nom i configuració bàsica: Es defineix un node anomenat `master` amb 2 CPUs, 2 GB de RAM, i una IP privada (10.0.0.10).
- Imatge base: Utilitza la box de Vagrant [learnk8s/networking](https://www.vagrantbox.com/learnk8s/networking) (versió 3.0.0) que porta la versió 1.20.15 de Kubernetes

- Provisió del node:
 - Inicialitza el clúster de Kubernetes amb `kubeadm`.
 - Configura el fitxer `admin.conf` per al comandament `kubectl`.
 - Aplica el plugin de xarxa Flannel per gestionar el tràfic entre Pods.
 - Configura paràmetres addicionals com la IP del `kubelet`.

Configuració dels Nodes Treballadors

- Noms dinàmics: Els nodes s'anomenen `worker1` i `worker2`.
- IP Privades: Assigna IPs privades seqüencials (10.0.0.11, 10.0.0.12).
- Provisió dels nodes:
 - Executa un script que uneix cada node al clúster utilitzant el fitxer `join.sh`, generat pel node màster.
 - Configura el `kubelet` perquè utilitzi la IP correcta.
 - Habilita *Docker* per executar els contenadors.

Script de Provisió del Màster

El bloc `$provision_master` és un script de shell que fa:

1. Inicialització del Clúster: Utilitza `kubeadm init` per crear el node `master` i defineix la xarxa del clúster (10.244.0.0/16).
2. Generació de `join.sh`: Guarda el comandament necessari perquè els nodes treballadors s'uneixin al clúster.
3. Configuració del client `kubectl`: Copia les credencials necessàries per interactuar amb el clúster.
4. Implementació de *Flannel*: Aplica el fitxer de configuració de *Flannel* com a plugin de xarxa.
5. Configuració del `kubelet`: Assigna explícitament la IP del node.

Característiques Addicionals

- **Plugins de Xarxa:** Utilitza *Flannel* com a solució de xarxa per permetre la comunicació entre els Pods.
- **Escalabilitat i Modularitat:** La configuració és escalable, ja que permet afegir fàcilment més nodes treballadors.

Per obtenir el nostre laboratori primer instal·lem VirtualBox, en el nostre cas la màquina amfitriona és un Debian, per tant es troba al repositori oficial de Debian i només cal triar la versió correcta per instal·lar-ho, que serà la 7.0 per tal que sigui compatible amb el nostre VagrantFile.

Opcionalment podem instal·lar també les `virtualbox-guest-additions-iso`, que és un bon complement per VirtualBox i tenim funcions avançades per compartir dispositius amb l'amfitrió, directoris, etc.

Per veure les disponibles podem fer `sudo apt search virtualbox-`

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices$ sudo apt search virtualbox-
Sorting... Done
Full Text Search... Done
boinc-virtualbox/stable 7.20.5+dfsg-1.1 amd64
  metapackage for virtualbox-savvy projects

virtualbox-6.1/unknown,now 6.1.50-161033-Debian~bookworm amd64 [residual-config]
  Oracle VM VirtualBox

virtualbox-7.0/unknown,now 7.0.22-165102-Debian~bookworm amd64 [installed]
  Oracle VM VirtualBox

virtualbox-7.1/unknown,now 7.1.4-165100-Debian~bookworm amd64 [residual-config]
  Oracle VirtualBox

virtualbox-guest-additions-iso/stable 7.0.6-1 all
  guest additions iso image for VirtualBox
```

Per instal·lar-ho fem: `sudo apt install virtualbox-7.0`

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices$ sudo apt install virtualbox-7.0
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Recommended packages:
  linux-image
The following NEW packages will be installed:
  virtualbox-7.0
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 92.6 MB of archives.
After this operation, 219 MB of additional disk space will be used.
Get:1 https://download.virtualbox.org/virtualbox/debian bookworm/contrib amd64 virtualbox-7.0 amd64 7.0.22-165102-Debian~bookworm [92.6 MB]
Fetched 92.6 MB in 1s (69.9 MB/s)
Preconfiguring packages ...
Selecting previously unselected package virtualbox-7.0.
(Reading database ... 260313 files and directories currently installed.)
Preparing to unpack .../virtualbox-7.0_7.0.22-165102-Debian~bookworm_amd64.deb ...
Unpacking virtualbox-7.0 (7.0.22-165102-Debian~bookworm) ...
Setting up virtualbox-7.0 (7.0.22-165102-Debian~bookworm) ...
addgroup: The group 'vboxusers' already exists as a system group. Exiting.
Processing triggers for desktop-file-utils (0.26-1) ...
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for shared-mime-info (2.2-1) ...
Processing triggers for mailcap (3.70+nmul) ...
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices$
```

Un cop tenim VirtualBox instal·lat, instal·lem Vagrant si no el teníem del laboratori anterior amb `sudo apt install vagrant`

Finalment, executem `vagrant up`

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices/mv-kubernetes-
vagrant-install-ubuntu$ vagrant destroy -f
==> worker2: Forcing shutdown of VM...
==> worker2: Destroying VM and associated drives...
==> worker1: Forcing shutdown of VM...
==> worker1: Destroying VM and associated drives...
==> master: Forcing shutdown of VM...
==> master: Destroying VM and associated drives...
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices/mv-kubernetes-
vagrant-install-ubuntu$ vagrant up
Bringing machine 'master' up with 'virtualbox' provider...
Bringing machine 'worker1' up with 'virtualbox' provider...
Bringing machine 'worker2' up with 'virtualbox' provider...
==> master: Importing base box 'learnk8s/networking'...
==> master: Matching MAC address for NAT networking...
```

```

==> master: Checking if box 'learnk8s/networking' version '3.0.0' is up to
date...
==> master: Setting the name of the VM: master
==> master: Clearing any previously set network interfaces...
==> master: Preparing network interfaces based on configuration...
    master: Adapter 1: nat
    master: Adapter 2: hostonly
==> master: Forwarding ports...
    master: 22 (guest) => 2222 (host) (adapter 1)
==> master: Running 'pre-boot' VM customizations...
==> master: Booting VM...
==> master: Waiting for machine to boot. This may take a few minutes...
    master: SSH address: 127.0.0.1:2222
    master: SSH username: vagrant
    master: SSH auth method: private key
==> master: Machine booted and ready!
==> master: Checking for guest additions in VM...
    master: The guest additions on this VM do not match the installed version
of
    master: VirtualBox! In most cases this is fine, but in rare cases it can
    master: prevent things such as shared folders from working properly. If
you see
    master: shared folder errors, please make sure the guest additions within
the
    master: virtual machine match the version of VirtualBox you have
installed on
    master: your host and reload your VM.
    master:
    master: Guest Additions Version: 6.1.6
    master: VirtualBox Version: 7.0
==> master: Setting hostname...
==> master: Configuring and enabling network interfaces...
==> master: Mounting shared folders...
    master: /vagrant => /home/sergio/OneDrive/TFG/Project/microservices/mv-
kubernetes-vagrant-install-ubuntu
==> master: Running provisioner: shell...
    master: Running: inline script
    master: [WARNING Service-Docker]: docker service is not enabled,
please run 'systemctl enable docker.service'
    master: [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the
Docker cgroup driver. The recommended driver is "systemd". Please follow the
guide at https://kubernetes.io/docs/setup/cri/
    master: Reading package lists...
    master: Building dependency tree...
    master: Reading state information...
    master: The following NEW packages will be installed:
    master:  bash-completion
    master: 0 upgraded, 1 newly installed, 0 to remove and 207 not upgraded.
    master: Need to get 178 kB of archives.
    master: After this operation, 1,523 kB of additional disk space will be
used.
    master: Get:1 http://archive.ubuntu.com/ubuntu focal/main amd64 bash-
completion all 1:2.10-1ubuntu1 [178 kB]
    master: dpkg-preconfigure: unable to re-open stdin: No such file or
directory
    master: Fetched 178 kB in 0s (755 kB/s)
    master: Selecting previously unselected package bash-completion.
(Reading database ... 44917 files and directories currently installed.)

```

```

master:   Preparing to unpack .../bash-completion_1%3a2.10-
1ubuntu1_all.deb ...
master: Unpacking bash-completion (1:2.10-1ubuntu1) ...
master: Setting up bash-completion (1:2.10-1ubuntu1) ...
master: Processing triggers for man-db (2.9.1-1) ...
master: Environment="KUBELET_EXTRA_ARGS=--node-ip=10.0.0.10"
master: Podsecuritypolicy.policy/psp.flannel.unprivileged created
master: Warning: rbac.authorization.k8s.io/v1beta1 ClusterRole is
deprecated in v1.17+, unavailable in v1.22+; use
rbac.authorization.k8s.io/v1 ClusterRole
master: clusterrole.rbac.authorization.k8s.io/flannel created
master: Warning: rbac.authorization.k8s.io/v1beta1 ClusterRoleBinding is
deprecated in v1.17+, unavailable in v1.22+; use
rbac.authorization.k8s.io/v1 ClusterRoleBinding
master: clusterrolebinding.rbac.authorization.k8s.io/flannel created
master: serviceaccount/flannel created
master: configmap/kube-flannel-cfg created
master: daemonset.apps/kube-flannel-ds created
master: Created symlink /etc/systemd/system/multi-
user.target.wants/docker.service → /lib/systemd/system/docker.service.
==> worker1: Importing base box 'learnk8s/networking'...
==> worker1: Matching MAC address for NAT networking...
==> worker1: Checking if box 'learnk8s/networking' version '3.0.0' is up to
date...
==> worker1: Setting the name of the VM: worker1
==> worker1: Fixed port collision for 22 => 2222. Now on port 2200.
==> worker1: Clearing any previously set network interfaces...
==> worker1: Preparing network interfaces based on configuration...
worker1: Adapter 1: nat
worker1: Adapter 2: hostonly
==> worker1: Forwarding ports...
worker1: 22 (guest) => 2200 (host) (adapter 1)
==> worker1: Running 'pre-boot' VM customizations...
==> worker1: Booting VM...
==> worker1: Waiting for machine to boot. This may take a few minutes...
worker1: SSH address: 127.0.0.1:2200
worker1: SSH username: vagrant
worker1: SSH auth method: private key
==> worker1: Machine booted and ready!
==> worker1: Checking for guest additions in VM...
worker1: The guest additions on this VM do not match the installed version
of
worker1: VirtualBox! In most cases this is fine, but in rare cases it
can
worker1: prevent things such as shared folders from working properly. If
you see
worker1: shared folder errors, please make sure the guest additions
within the
worker1: virtual machine match the version of VirtualBox you have
installed on
worker1: your host and reload your VM.
worker1:
worker1: Guest Additions Version: 6.1.6
worker1: VirtualBox Version: 7.0
==> worker1: Setting hostname...
==> worker1: Configuring and enabling network interfaces...
==> worker1: Mounting shared folders...

```

```

worker1: /vagrant =>
/home/sergio/OneDrive/TFG/Project/microservices/mv-kubernetes-vagrant-
install-ubuntu
==> worker1: Running provisioner: shell...
worker1: Running: inline script
worker1: [preflight] Running pre-flight checks
worker1: [WARNING Service-Docker]: docker service is not enabled,
please run 'systemctl enable docker.service'
worker1: [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the
Docker cgroup driver. The recommended driver is "systemd". Please follow the
guide at https://kubernetes.io/docs/setup/cri/
worker1: [preflight] Reading configuration from the cluster...
worker1: [preflight] FYI: You can look at this config file with 'kubectl
-n kube-system get cm kubeadm-config -o yaml'
worker1: [kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
worker1: [kubelet-start] Writing kubelet environment file with flags to
file "/var/lib/kubelet/kubeadm-flags.env"
worker1: [kubelet-start] Starting the kubelet
worker1: [kubelet-start] Waiting for the kubelet to perform the TLS
Bootstrap...
worker1:
worker1: This node has joined the cluster:
worker1: * Certificate signing request was sent to apiserver and a
response was received.
worker1: * The Kubelet was informed of the new secure connection details.
worker1:
worker1: Run 'kubectl get nodes' on the control-plane to see this node
join the cluster.
worker1:
worker1: Environment="KUBELET_EXTRA_ARGS=--node-ip=10.0.0.11"
worker1: Created symlink /etc/systemd/system/multi-
user.target.wants/docker.service → /lib/systemd/system/docker.service.
==> worker2: Importing base box 'learnk8s/networking'...
==> worker2: Matching MAC address for NAT networking...
==> worker2: Checking if box 'learnk8s/networking' version '3.0.0' is up to
date...
==> worker2: Setting the name of the VM: worker2
==> worker2: Fixed port collision for 22 => 2222. Now on port 2201.
==> worker2: Clearing any previously set network interfaces...
==> worker2: Preparing network interfaces based on configuration...
worker2: Adapter 1: nat
worker2: Adapter 2: hostonly
==> worker2: Forwarding ports...
worker2: 22 (guest) => 2201 (host) (adapter 1)
==> worker2: Running 'pre-boot' VM customizations...
==> worker2: Booting VM...
==> worker2: Waiting for machine to boot. This may take a few minutes...
worker2: SSH address: 127.0.0.1:2201
worker2: SSH username: vagrant
worker2: SSH auth method: private key
==> worker2: Machine booted and ready!
==> worker2: Checking for guest additions in VM...
worker2: The guest additions on this VM do not match the installed version
of
worker2: VirtualBox! In most cases this is fine, but in rare cases it
can

```

```

worker2: prevent things such as shared folders from working properly. If
you see
worker2: shared folder errors, please make sure the guest additions
within the
worker2: virtual machine match the version of VirtualBox you have
installed on
worker2: your host and reload your VM.
worker2:
worker2: Guest Additions Version: 6.1.6
worker2: VirtualBox Version: 7.0
==> worker2: Setting hostname...
==> worker2: Configuring and enabling network interfaces...
==> worker2: Mounting shared folders...
worker2: /vagrant =>
/home/sergio/OneDrive/TFG/Project/microservices/mv-kubernetes-vagrant-
install-ubuntu
==> worker2: Running provisioner: shell...
worker2: Running: inline script
worker2: [preflight] Running pre-flight checks
worker2: [WARNING Service-Docker]: docker service is not enabled,
please run 'systemctl enable docker.service'
worker2: [WARNING IsDockerSystemdCheck]: detected "cgroupfs" as the
Docker cgroup driver. The recommended driver is "systemd". Please follow the
guide at https://kubernetes.io/docs/setup/cri/
worker2: [preflight] Reading configuration from the cluster...
worker2: [preflight] FYI: You can look at this config file with 'kubectl
-n kube-system get cm kubeadm-config -o yaml'
worker2: [kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
worker2: [kubelet-start] Writing kubelet environment file with flags to
file "/var/lib/kubelet/kubeadm-flags.env"
worker2: [kubelet-start] Starting the kubelet
worker2: [kubelet-start] Waiting for the kubelet to perform the TLS
Bootstrap...
worker2:
worker2: This node has joined the cluster:
worker2: * Certificate signing request was sent to apiserver and a
response was received.
worker2: * The Kubelet was informed of the new secure connection details.
worker2:
worker2: Run 'kubectl get nodes' on the control-plane to see this node
join the cluster.
worker2:
worker2: Environment="KUBELET_EXTRA_ARGS=--node-ip=10.0.0.12"
worker2: Created symlink /etc/systemd/system/multi-
user.target.wants/docker.service → /lib/systemd/system/docker.service.

```

En finalitzar si fem `vagrant status` podem veure les tres màquines virtuals creades:

```

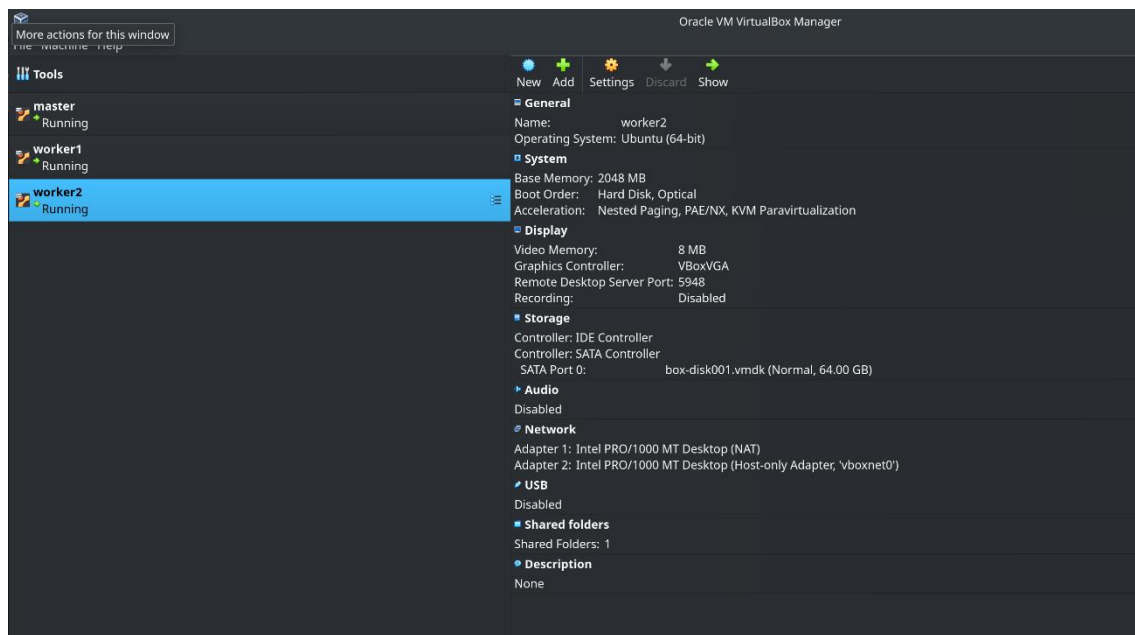
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices/mv-kubernetes-vagrant-install-ubuntu$ vagrant status
Current machine states:

master           running (virtualbox)
worker1          running (virtualbox)
worker2          running (virtualbox)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run 'vagrant status NAME'.

```

Des de la GUI del VirtualBox podem accedir:



Per entrar a qualsevol dels nodes amb SSH farem `vagrant ssh <node>`, per exemple entrem al node `master` on tenim el control-plane instal·lat per executar comandes `kubectl` que controlen el cluster, com ara `kubectl get nodes -o wide` per veure l'estat dels nodes del cluster:

```
sergio@sergio-xps-deb:~/OneDrive/TFG/Project/microservices/mv-kubernetes-vagrant-install-ubuntu$ vagrant ssh master
Welcome to Ubuntu 20.04 LTS (GNU/Linux 5.4.0-29-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Thu 02 Jan 2025 11:07:39 AM UTC

System load:  0.14           Users logged in:      0
Usage of /:   6.7% of 61.31GB IPv4 address for cni0: 10.244.0.1
Memory usage: 38%           IPv4 address for docker0: 172.17.0.1
Swap usage:   0%            IPv4 address for eth0: 10.0.2.15
Processes:   154            IPv4 address for eth1: 10.0.0.10

 * Introducing self-healing high availability clusters in MicroK8s.
   Simple, hardened, Kubernetes for production, from RaspberryPi to DC.

   https://microk8s.io/high-availability

213 updates can be installed immediately.
112 of these updates are security updates.
To see these additional updates run: apt list --upgradable

This system is built by the Bento project by Chef Software
More information can be found at https://github.com/chef/bento
vagrant@master:~$ kubectl get nodes -o wide
NAME        STATUS    ROLES                  AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
master      Ready     control-plane,master   7m16s   v1.20.4   10.0.0.10     <none>         Ubuntu 20.04 LTS     5.4.0-29-generic  docker://19.3.8
worker1     Ready     <none>                 6m15s   v1.20.4   10.0.0.11     <none>         Ubuntu 20.04 LTS     5.4.0-29-generic  docker://19.3.8
worker2     Ready     <none>                 5m20s   v1.20.4   10.0.0.12     <none>         Ubuntu 20.04 LTS     5.4.0-29-generic  docker://19.3.8
```

Ja tenim el nostre cluster de Kubernetes llest per desplegar aplicacions.

```
vagrant@master:~$ kubectl version
Client Version: version.Info{Major:"1", Minor:"20", GitVersion:"v1.20.4", GitCommit:"e87da0bd6e03c3fea7933c4b5263d151aafd07c", GitTreeState:"clean", BuildDate:"2021-02-18T16:12:00Z", GoVersion:"go1.15.8", Compiler:"gc", Platform:"linux/amd64"}
Server Version: version.Info{Major:"1", Minor:"20", GitVersion:"v1.20.15", GitCommit:"8f1e5b0b9729a899b8df86249b56e2c74aebc55", GitTreeState:"clean", BuildDate:"2022-01-19T17:23:01Z", GoVersion:"go1.15.15", Compiler:"gc", Platform:"linux/amd64"}
vagrant@master:~$
```

1.3. Instal·lació de HELM en MiniKube

Helm es pot instal·lar de diferents maneres, ja sigui a partir de la versió binària o mitjançant un script.

Per instal·lar a partir de la versió binària:

- Descarrega la versió desitjada des del lloc oficial de Helm.
- Descomprimeix la carpeta.
- Mou els fitxers al destí desitjat: `/usr/local/bin/helm`.

Per instal·lar mitjançant un script executa les següents comandes per descarregar l'script i canviar els permisos.

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
```

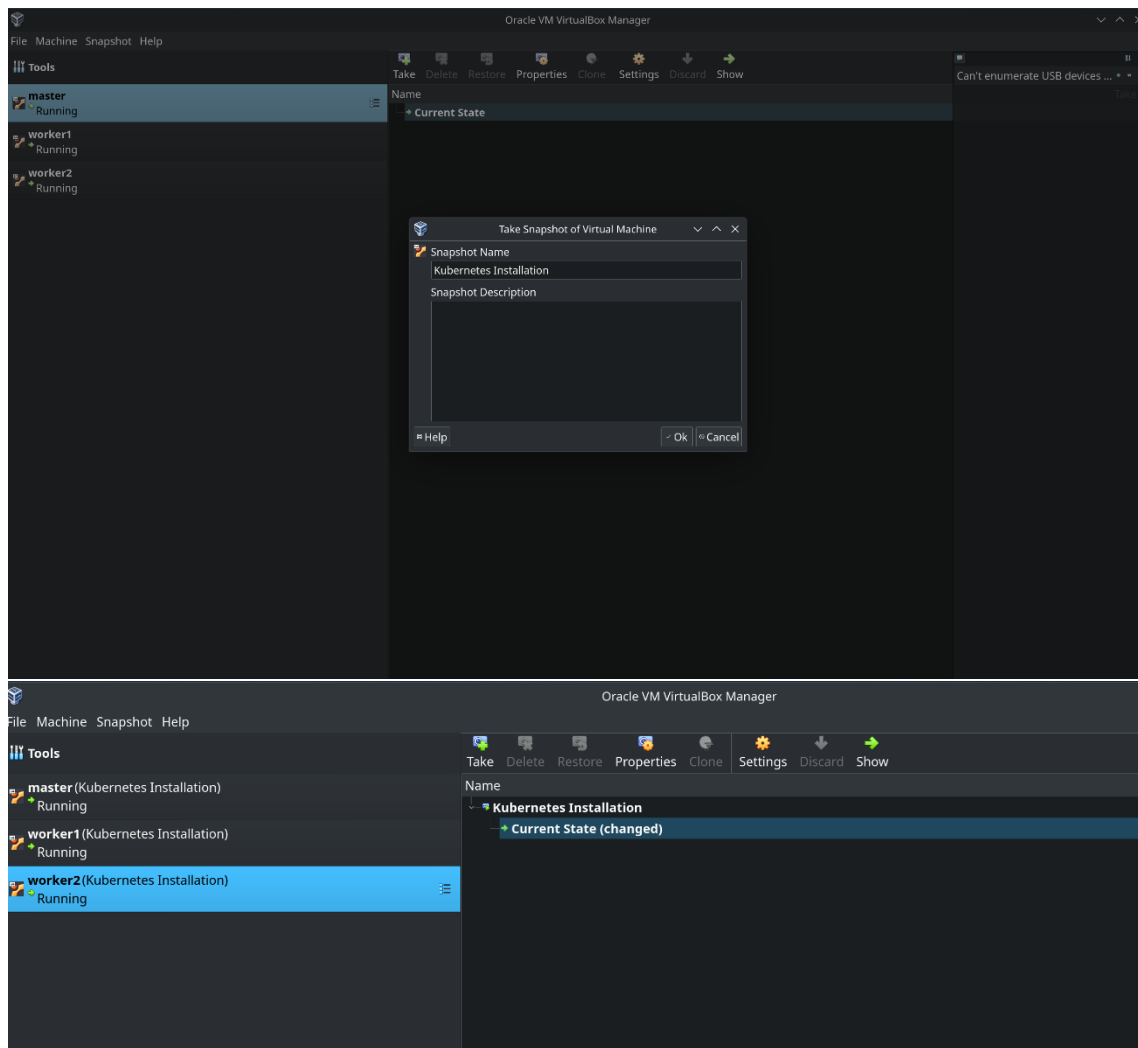
També el podem instal·lar directament amb el gestor de paquets del sistema operatiu: Homebrew (macOS), Chocolatey o Scoop (Windows), Apt (Debian based), Yum (RedHat Based)...

Ho farem amb l'opció d'scripts i ho deixarem tot en `/vagrant/helm` per poder tenir control des del node amfitrió, més tard possarem aquí les aplicacions de HELM que versionarem amb git:

```
vagrant@master:/vagrant/helm$ curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
mod 700 get_helm.sh

./get_helm.sh vagrant@master:/vagrant/helm$
vagrant@master:/vagrant/helm$ chmod 700 get_helm.sh
vagrant@master:/vagrant/helm$
vagrant@master:/vagrant/helm$ ./get_helm.sh
Downloading https://get.helm.sh/helm-v3.16.4-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
vagrant@master:/vagrant/helm$
```

Abans de continuar i per tenir un backup en cas de fallida, farem snapshot de les màquines virtuals des de VirtualBox:



1.3.1 Instal·lació d'apps a Kubernetes amb Charts HELM

Helm és una eina de gestió de paquets per a Kubernetes que utilitza "charts" (paquets de configuració) per desplegar aplicacions. Els fitxers `values.yaml` i `Chart.yaml` són dos fitxers essencials en un chart Helm que permeten gestionar la configuració i les metadades de l'aplicació que s'instal·la al cluster Kubernetes. Dins del directori `templates` de HELM es defineixen els components bàsics de l'aplicació que correrà a Kubernetes. Els charts es poden inicialitzar en fer `helm create <nom_app>` i es poden aconseguir també de repositoris oficial o desde github, on tenim per exemple els repositoris de bitnami amb multitud d'aplicacions pre-configurades.

Exemple de Helm Chart:

```
myapp/
├── Chart.yaml
├── values.yaml
├── charts/
└── templates/
    ├── deployment.yaml
    ├── configmap.yaml
    ├── hpa.yaml
    ├── service.yaml
    ├── secrets.yaml
    ├── ingress.yaml
    └── storage.yaml
```

Chart.yaml

El fitxer `Chart.yaml` és el fitxer de metadades del chart. Conté informació bàsica sobre el chart com el seu nom, versió, descripció, i dependències. Aquest fitxer és necessari per a qualsevol chart Helm i s'utilitza per identificar el chart i la seva configuració general.

values.yaml

El fitxer `values.yaml` és el lloc on es defineixen els valors per defecte que es poden utilitzar a tot el chart. És una manera senzilla de proporcionar configuracions personalitzades per als recursos que s'instal·len al Kubernetes. Els valors definits en aquest fitxer poden ser substituïts mitjançant arguments de la línia de comandes o mitjançant fitxers d'*override* quan s'instal·la o actualitza un chart.

templates/deployment.yaml

Aquest fitxer defineix el desplegament de l'aplicació, és una plantilla que defineix un recurs `Deployment` de Kubernetes. Un `Deployment` s'encarrega de gestionar la creació, actualització i escalat d'un conjunt de `Pods` en un cluster de Kubernetes. Els `Deployment` permeten administrar l'estat desitjat de l'aplicació, incloent el nombre de rèpliques i l'estratègia de desplegament, i garantir que el nombre de `Pods` executant l'aplicació es mantingui constant.

`templates/configmap.yaml`

S'utilitza per crear un `ConfigMap` de Kubernetes, que emmagatzema informació de configuració en forma de claus i valors. Aquesta configuració es pot utilitzar dins de contenidors sense necessitat de codificar-la al propi contenidor, permetent una major flexibilitat i facilitat d'actualització.

`templates/hpa.yaml`

El fitxer `hpa.yaml` serveix per definir un `HorizontalPodAutoscaler` (HPA) de Kubernetes. Aquesta eina ajusta automàticament el nombre de rèpliques d'un `Deployment`, `ReplicaSet` o `StatefulSet` en funció de mètriques com l'ús de CPU, memòria o altres valors personalitzats. És útil per gestionar càrregues variables i assegurar que els recursos són utilitzats eficientment.

`templates/service.yaml`

Un `Service` és una manera d'exposar una aplicació o un conjunt de contenidors (Pods) dins del cluster de Kubernetes de manera consistent i fiable. Els `Services` proporcionen una abstracció per accedir als Pods d'una aplicació, permetent que els clients es comuniquin amb aquests Pods de forma consistent, fins i tot si els Pods es creen o eliminen dinàmicament. Els `Services` poden ser exposats de diverses maneres, com ara internament dins del clúster o externament.

`templates/secrets.yaml`

Els secrets són objectes utilitzats per a emmagatzemar i gestionar informació sensible com ara credencials, contrasenyes, claus d'API, certificats, etc. En aquest context, aquest fitxer es genera dinàmicament a partir dels valors definits al fitxer `values.yaml`.

`templates/ingress.yaml`

En Kubernetes es fa servir per configurar un recurs `Ingress`, que controla com les sol·licituds externes (normalment HTTP o HTTPS) són enrutades a les aplicacions o serveis que estan funcionant dins del teu cluster de Kubernetes. Aquest recurs és essencial per exposar les teves aplicacions de manera controlada i flexible, utilitzant un únic punt d'entrada (un sol URL) per accedir a diferents serveis dins del teu cluster.

`templates/storage.yaml`

Les peticions d'espai d'emmagatzematge dinàmic es defineixen aquí on es vinculen amb `PersistentVolumes` (PV) al clúster i s'assignen als Pods amb `PersistentVolumeClaims` (PVC). Això assegura que les aplicacions poden disposar de recursos d'emmagatzematge adequats, separats de la vida útil dels Pods que les executen.

Creació HELM chart per a LAMP

Per muntar una aplicació **LAMP** (Linux, Apache, MySQL, PHP) al clúster Kubernetes utilitzant Helm, farem aquests passos:

Preparar el clúster i verificar Helm

Abans de començar, ens hem d'assegurar que:

- El clúster Kubernetes està actiu i funcional.
- Helm està instal·lat i configurat per interactuar amb el teu clúster.

Comprovació del clúster:

```
kubectl cluster-info
```

```
vagrant@master:~$ kubectl cluster-info
Kubernetes control plane is running at https://10.0.0.10:6443
KubeDNS is running at https://10.0.0.10:6443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
vagrant@master:~$
```

Comprovació de Helm:

```
helm version
```

```
vagrant@master:/vagrant/helm$ helm version
version.BuildInfo{Version:"v3.16.4", GitCommit:"7877b45b63f95635153b29a42c0c2f4273ec45ca", GitTreeState:"clean", GoVersion:"go1.22.7"}
vagrant@master:/vagrant/helm$
```

Crear un Helm Chart per a LAMP

Generem un nou Helm chart base per a LAMP:

```
helm create lamp
```

El directori lamp contindrà la configuració bàsica del chart.

```
vagrant@master:/vagrant/helm$ helm create lamp
Creating lamp
vagrant@master:/vagrant/helm$ ls -l lamp
total 20
drwxr-xr-x 1 vagrant vagrant 4096 Jan  2 12:07 charts
-rw-r--r-- 1 vagrant vagrant 1140 Jan  2 12:07 Chart.yaml
drwxr-xr-x 1 vagrant vagrant 4096 Jan  2 12:07 templates
-rw-r--r-- 1 vagrant vagrant 4290 Jan  2 12:07 values.yaml
vagrant@master:/vagrant/helm$
```

Configurar els serveis de LAMP

Editarem o creem els fitxers del Helm Chart per incloure cada component de LAMP:

Creació del fitxer `deployment.yaml`

Aquest fitxer estarà dins del directori `templates` i conté configuracions de Deployment per a tres components d'una aplicació LAMP (Apache, MySQL i PHP) en Kubernetes. Cada Deployment garanteix que un nombre desitjat de rèpliques d'una aplicació estiguin en execució i també gestiona les actualitzacions i l'escalabilitat automàticament.

En aquest fitxer tenim tres Deployments independents per a cada component:

1. **Apache:** Serveix com a servidor web.
2. **MySQL:** Proporciona la base de dades, utilitzant secrets per emmagatzemar informació sensible com la contrasenya.
3. **PHP:** Gestiona el processament PHP, amb configuracions per controlar els límits de memòria i el temps d'execució.

Cada Deployment garanteix que els contenidors per Apache, MySQL i PHP s'executin amb les configuracions corresponents i es poden escalar de manera independent. També es fa servir emmagatzematge persistent per a MySQL per mantenir les dades entre reinicis de Pods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-apache
  labels:
    app: {{ .Chart.Name }}
    component: apache
  namespace: {{ .Release.namespace }}
spec:
  replicas: {{ .Values.autoscaling.minReplicas }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
      component: apache
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
        component: apache
    spec:
      containers:
        - name: apache
          image: {{ .Values.apache.image }}
          ports:
            - containerPort: {{ .Values.apache.port }}
```

```

---

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-mysql
  labels:
    app: {{ .Chart.Name }}
    component: mysql
  namespace: {{ .Release.namespace }}
spec:
  replicas: {{ .Values.autoscaling.minReplicas }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
      component: mysql
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
        component: mysql
    spec:
      containers:
        - name: mysql
          image: {{ .Values.mysql.image }}
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-mysql-secrets
                  key: mysql-root-password
            - name: MYSQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-mysql-secrets
                  key: mysql-database
            - name: MYSQL_USER
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-mysql-secrets
                  key: mysql-user
            - name: MYSQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: {{ .Release.Name }}-mysql-secrets
                  key: mysql-password
          ports:
            - containerPort: {{ .Values.mysql.port }}

```

```

    volumeMounts:
      - name: mysql-data
        mountPath: /var/lib/mysql
      - name: mysql-config
        mountPath: /etc/mysql/conf.d/my.cnf
        subPath: my.cnf
    securityContext:
      runAsUser: 1000
  volumes:
    - name: mysql-data
      persistentVolumeClaim:
        claimName: {{ .Release.Name }}-mysql-pvc
    - name: mysql-config
      configMap:
        name: {{ .Release.Name }}-mysql-config
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-php
  labels:
    app: {{ .Chart.Name }}
    component: php
  namespace: {{ .Release.namespace }}
spec:
  replicas: {{ .Values.autoscaling.minReplicas }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
      component: php
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
        component: php
    spec:
      containers:
        - name: php
          image: {{ .Values.php.image }}
          ports:
            - containerPort: {{ .Values.php.port }}
          env:
            - name: PHP_MEMORY_LIMIT
              value: "{{ .Values.php.memoryLimit }}"
            - name: PHP_MAX_EXECUTION_TIME
              value: "{{ .Values.php.maxExecutionTime }}"
          volumeMounts:

```

```

- name: php-data
  mountPath: /var/www/html
  securityContext:
    runAsUser: 1000
volumes:
- name: php-data
  persistentVolumeClaim:
    claimName: {{ .Release.Name }}-php-pvc

```

Anem a veure els detalls de cada secció:

Deployment per Apache

- **metadata:** Defineix el nom del Deployment i les etiquetes (**labels**) per identificar el recurs.
- **spec:**
 - **replicas:** El nombre de rèpliques que ha d'estar executant-se, en aquest cas s'assigna amb `{{ .Values.autoscaling.minReplicas }}` que és 1.
 - **selector:** Definiu els criteris per identificar quins Pods gestionarà aquest Deployment, mitjançant les etiquetes.
 - **template:** La plantilla per als Pods. Inclou el nom de les etiquetes i els containers a executar:
 - **containers:** El contenidor utilitzarà la imatge definida en `{{ .Values.apache.image }}` i escoltarà al port `80` `{{ .Values.apache.port }}`.

Deployment per MySQL

- **metadata:** Com en l'anterior, es defineix el nom i les etiquetes.
- **spec:**
 - **replicas:** El nombre de rèpliques s'assigna amb `{{ .Values.autoscaling.minReplicas }}` que és 1.
 - **selector:** Especifiquem com es trobaran els **Pods** a gestionar.
 - **template:** Descriu el pod que es crearà per executar MySQL:
 - **containers:**
 - El contenidor de MySQL utilitza la imatge especificada a `{{ .Values.mysql.image }}` i escoltarà al port `3306` definit en `{{ .Values.mysql.port }}`.
 - **env:** Les variables d'entorn per a la configuració de MySQL s'afegeixen des de **secrets**, com `MYSQL_ROOT_PASSWORD`.
 - **volumeMounts:** El contenidor monta el volum persistent a `/var/lib/mysql` per emmagatzemar la base de dades. A més també especifica el volum pel ConfigMap `/etc/mysql/conf.d/my.cnf`
 - **volumes:** Es defineix el volum com una **PersistentVolumeClaim (PVC)** per garantir l'emmagatzematge de dades de MySQL.

Deployment per PHP

- **metadata:** Definiu el nom i etiquetes per aquest Deployment de PHP.
- **spec:**
 - **replicas:** En aquest cas també s'assigna amb `{{ .Values.autoscaling.minReplicas }}` que és 1..
 - **selector:** Per identificar els Pods a gestionar.
 - **template:** Es defineix el pod per a PHP:
 - **containers:** El contenidor PHP utilitza la imatge definida en `{{ .Values.php.image }}` i escoltarà al port **9000** definit en `{{ .Values.php.port }}`.
 - **env:** S'estableixen les variables d'entorn per a la configuració de PHP com el límit de memòria i el temps màxim d'execució.
 - **volumeMounts:** El contenidor monta el volum persistent a `/var/www/html` per emmagatzemar la base de dades.
 - **volumes:** Es defineix el volum com una **PersistentVolumeClaim (PVC)** per garantir l'emmagatzematge de dades de PHP.

Creació del fitxer configmap.yaml

El **ConfigMap** és un recurs de Kubernetes que permet emmagatzemar dades de configuració en format clau-valor. El propòsit principal d'aquest **ConfigMap** és proporcionar configuració personalitzada per a PHP dins del contenidor de l'aplicació LAMP (Linux, Apache, MySQL, PHP). Aquesta configuració es passa a PHP a través d'un fitxer `php.ini`, el qual es pot muntar dins del contenidor del servei PHP en Kubernetes. Així, es permet que l'entorn PHP sigui configurat segons les necessitats específiques de l'usuari, com els límits de memòria, els temps d'execució i la gestió d'errors.

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-php-config
  namespace: {{ .Values.namespace }}
data:
  php.ini: |
    memory_limit = {{ .Values.php.memoryLimit }}
    max_execution_time = {{ .Values.php.maxExecutionTime }}
    upload_max_filesize = {{ .Values.php.uploadMaxFilesize }}
    post_max_size = {{ .Values.php.postMaxSize }}
    error_reporting = {{ .Values.php.iniSettings.error_reporting }}
    display_errors = {{ .Values.php.iniSettings.display_errors }}
    log_errors = {{ .Values.php.iniSettings.log_errors }}
```

```

    opcache_enable = {{ .Values.php.iniSettings.opcache_enable }}

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-mysql-config
  namespace: {{ .Values.namespace }}
data:
  my.cnf: |
    [mysqld]
    innodb_use_native_aio = 0

```

ConfigMap de PHP

Cada línia dins del fitxer `php.ini` estableix una directiva de configuració per PHP:

- `memory_limit`: Defineix la quantitat màxima de memòria que PHP pot utilitzar. El valor es carrega des de `{{ .Values.php.memoryLimit }}`, que prové del fitxer `values.yaml`.
- `max_execution_time`: Defineix el temps màxim en segons que un script PHP pot estar en execució abans que es cancel·li. Es carrega des de `{{ .Values.php.maxExecutionTime }}`.
- `upload_max_filesize`: Defineix la mida màxima dels fitxers que es poden carregar a PHP. S'estableix mitjançant `{{ .Values.php.uploadMaxFilesize }}`.
- `post_max_size`: Defineix la mida màxima de les dades de la sol·licitud que poden ser enviades a PHP. Es carrega des de `{{ .Values.php.postMaxSize }}`.
- `error_reporting`: Configura quins errors PHP ha de reportar. Es carrega des de `{{ .Values.php.iniSettings.error_reporting }}`.
- `display_errors`: Estableix si els errors han de ser mostrats directament als usuaris. S'obté des de `{{ .Values.php.iniSettings.display_errors }}`.
- `log_errors`: Configura si els errors han de ser registrats en un arxiu de log. Es carrega des de `{{ .Values.php.iniSettings.log_errors }}`.
- `opcache_enable`: Habilita o deshabilita l'ús d'OPcache, un sistema de cache per optimitzar el rendiment de PHP. Es carrega des de `{{ .Values.php.iniSettings.opcache_enable }}`.

ConfigMap de MySQL

La configuració de MySQL s'estableix al fitxer `my.cnf`:

- `innodb_use_native_aio`: Habilita o deshabilita AIO (asynchronous I/O) per InnoDB, alguns tipus de filesystems no son compatibles amb AIO i el deshabilitem pel nostre laboratori. En un entorn real fariem servir filesystem com `ext4` o `xfs` que suporten I/O asíncrona. Es carrega des de `{{ .Values.mysql.mycnfSettings.innodb_use_native_aio }}`

Creació del fitxer hpa.yaml

Un HPA ajusta automàticament el nombre de rèpliques dels Pods d'un desplegament en funció de l'ús de recursos com la CPU o la memòria.

```
# Horizontal Pod Autoscaler for Apache
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-apache
  labels:
    app: {{ .Chart.Name }}
    component: apache
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-apache
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  targetCPUUtilizationPercentage:
    {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
---

# Horizontal Pod Autoscaler for MySQL
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: {{ .Release.Name }}-mysql
  labels:
    app: {{ .Chart.Name }}
    component: mysql
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-mysql
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  targetCPUUtilizationPercentage:
    {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
---

# Horizontal Pod Autoscaler for PHP
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
```

```

name: {{ .Release.Name }}-php
labels:
  app: {{ .Chart.Name }}
  component: php
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ .Release.Name }}-php
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
  targetCPUUtilizationPercentage:
    {{ .Values.autoscaling.targetCPUUtilizationPercentage }}

```

- `spec.scaleTargetRef`: Aquest camp especifica quina aplicació s'escala automàticament. En aquest cas, el tipus és `Deployment` i el nom es genera dinàmicament basant-se en la release i l'aplicació.
- `minReplicas` i `maxReplicas`: Aquests dos camps defineixen el rang de rèpliques entre les quals es pot ajustar el nombre de Pods:
 - `minReplicas`: El nombre mínim de rèpliques que el HPA mantindrà.
 - `maxReplicas`: El nombre màxim de rèpliques a què el HPA pot escalar.
- `Target CPU/Memory Utilization`: Aquest valor es carrega des de `values.yaml` i defineix el percentatge de CPU i Memòria respectivament al qual es vol que el sistema escali. Si l'ús de CPU o Memòria supera aquest valor, el HPA augmentarà el nombre de rèpliques.

Creació del fitxer `service.yaml`

El fitxer `services.yaml` s'utilitza per exposar els serveis dins de Kubernetes a través de Services. Aquest component defineix les regles per exposar les aplicacions (Apache, MySQL, PHP) com a serveis dins de Kubernetes, i permet accedir-hi des de dins o des de fora del clúster.

- Apache: Servei per a gestionar les peticions HTTP al port **80** (per defecte).
- MySQL: Servei per a la base de dades MySQL, que escolta al port **3306**.
- PHP: Servei per a gestionar el processament PHP a través de PHP-FPM, que escolta al port **9000**.

Tots aquests serveis utilitzen el tipus `ClusterIP`, el qual limita l'accés als serveis només dins del clúster de Kubernetes, i són dirigits als Pods corresponents mitjançant el selector basat en etiquetes. Aquests serveis són necessaris perquè Apache pugui comunicar-se amb PHP per processar les pàgines web, i per a MySQL perquè Apache (o PHP) pugui fer consultes a la base de dades.

```

# Apache Service
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-apache-service
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: apache
spec:
  type: NodePort
  ports:
    - port: {{ .Values.apache.port | default 80 }}
      targetPort: {{ .Values.apache.targetPort | default 80 }}
      nodePort: {{ .Values.apache.nodePort | default 30080 }}
  selector:
    app: {{ .Chart.Name }}
    component: apache

---

# MySQL Service
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-mysql-service
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: mysql
spec:
  type: ClusterIP
  ports:
    - port: {{ .Values.mysql.port | default 3306 }}
      targetPort: {{ .Values.mysql.targetPort | default 3306 }}
  selector:
    app: {{ .Chart.Name }}
    component: mysql

---

# PHP Service
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-php-service
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}

```

```

    component: php
spec:
  type: ClusterIP
  ports:
  - port: {{ .Values.php.port | default 9000 }}
    targetPort: {{ .Values.php.targetPort | default 9000 }}
  selector:
    app: {{ .Chart.Name }}
    component: php

```

Service Apache:

- **apiVersion:** Defineix la versió de l'API per a la creació del servei. **v1** és l'estàndard per als serveis en Kubernetes.
- **kind:** Especifica que aquest recurs és un **Service**.
- **metadata:**
 - **name:** El nom del servei es defineix dinàmicament amb el nom de l'alliberament (**Release.Name**).
 - **labels:** S'assignen etiquetes per identificar el servei, en aquest cas per associar-lo amb l'aplicació i el component Apache.
 - **spec:**
 - **type:** El tipus de servei és **NodePort**, la qual cosa significa que alguns ports son accessibles des de fora del clúster Kubernetes.
 - **ports:** Aquí es defineix el port públic del servei (Apache, per defecte **80**), i el port intern (**targetPort**) al qual es dirigeix el trànsit dins del contenidor Apache. **NodePort** fa que el servei sigui accessible des de fora del cluster.
 - **selector:** Identifica els **Pods** als quals aquest servei s'ha de dirigir, utilitzant les etiquetes corresponents al component Apache.

Service MySQL:

Aquest servei segueix la mateixa estructura que el servei Apache, amb diferències específiques per al component MySQL:

- **name:** El nom del servei serà dinàmicament creat amb el nom de l'alliberament i el component **mysql**.
- **port:** El port públic per MySQL és per defecte el port **3306**.
- **targetPort:** El port intern al qual MySQL escolta també és el **3306**.
- **selector:** Aquí també s'utilitzen etiquetes per identificar els **Pods** que executen MySQL.

Service PHP:

Aquest servei és per al component PHP. La configuració és similar als anteriors:

- **name:** Nom del servei creat dinàmicament amb el nom de l'alliberament i el component **PHP**.

- **ports:** El port públic és el **9000**, que és el port típic per a PHP-FPM (FastCGI Process Manager) per interactuar amb el servidor web.
- **selector:** Igual que amb els altres serveis, utilitza les etiquetes per identificar els Pods que executen PHP.

Creació del fitxer `secrets.yaml`

El fitxer `secrets.yaml` defineix un **Secret** en Kubernetes, el qual és un recurs utilitzat per emmagatzemar informació sensible com contrasenyes, claus API, o certificats de manera segura. En aquest cas, el fitxer s'utilitza per emmagatzemar les contrasenyes de MySQL (com la contrasenya de root i la contrasenya d'usuari) i encriptar-les en base64, una pràctica comuna per protegir dades sensibles dins de Kubernetes.

```
apiVersion: v1
kind: Secret
metadata:
  name: {{ .Release.Name }}-mysql-secrets
  namespace: {{ .Values.namespace }}
namespace: default
type: Opaque
data:
  mysql-database: {{ .Values.mysql.database }}
  mysql-user: {{ .Values.mysql.user }}
  mysql-root-password: {{ .Values.mysql.rootPassword }}
  mysql-password: {{ .Values.mysql.password }}
```

- **metadata**
 - **name:** `{{ .Release.Name }}-mysql-secret`: El nom del **Secret** es genera dinàmicament a partir del nom de la release de Helm, amb l'ajuda de `{{ .Release.Name }}`. Això assegura que el **Secret** té un nom únic basat en la release del Helm i facilita la seva gestió.
 - **type:** `Opaque`: El tipus de **Secret** és `Opaque`, que indica que el **Secret** conté dades arbitràries en format de base64, no estructurat (en contrast amb altres tipus com `kubernetes.io/dockerconfigjson` o `kubernetes.io/tls`).
- **data:** Conté les dades codificades en Base64.
 - Els valors per a `mysql-database`, `mysql-user`, `mysql-root-password` i `mysql-password` s'han de passar mitjançant `values.yaml` o durant la instal·lació del Chart amb Helm. El Helm utilitza la funció `b64enc` per codificar els valors de manera segura en Base64 abans d'emmagatzemar-los al **Secret**.

Creació del fitxer `ingress.yaml`

Aquest template servirà per crear un **Ingress** en Kubernetes quan el valor `ingress.enabled` està activat (`true`) en el fitxer `values.yaml`.

Aquesta secció defineix un recurs Kubernetes **Ingress**. A la línia `apiVersion:` `networking.k8s.io/v1`, es fa servir la versió adequada per treballar amb Ingress en Kubernetes. El nom de l'Ingress es genera dinàmicament amb `{{ include "lamp.fullname" . }}`, i també s'afegeixen etiquetes a l'Ingress amb `{{ include "lamp.labels" . | nindent 4 }}`, que inclou etiquetes per a la gestió de recursos dins del clúster.

Les anotacions es poden definir per afegir metadades addicionals a l'Ingress (com configuracions específiques per a controladors d'Ingress, per exemple, amb **NGINX Ingress Controller**). Si es defineixen a `values.yaml`, es transformen en format YAML mitjançant `toYaml` i s'indentaran correctament amb `nindent 4`.

```
{{- if .Values.ingress.enabled -}}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "lamp.fullname" . }}
  namespace: {{ .Values.namespace }}
  labels:
    {{- include "lamp.labels" . | nindent 4 }}
  {{- with .Values.ingress.annotations }}
  annotations:
    {{- toYaml . | nindent 4 }}
  {{- end }}
spec:
  rules:
    - host: "localhost"
      http:
        paths:
          - path: "/"
            pathType: Prefix
            backend:
              service:
                name: {{ include "lamp.fullname" $ }}
                port:
                  number: 80
  {{- end }}
```

- `rules` defineixen quines regles es faran servir per redirigir les sol·licituds HTTP que arriben a l'Ingress.
- `host: "localhost"` especifica que aquesta regla s'aplica quan les sol·licituds es realitzen a localhost.
- `http:` defineix el protocol HTTP i les seves rutes (`paths`).
 - `path: "/"` especifica que totes les sol·licituds que arriben a la ruta / seran redirigides. Potser podries afegir altres rutes si és necessari.
 - `pathType: Prefix` indica que qualsevol sol·licitud que comenci amb /

- serà redirigida, la qual cosa és útil per a una ruta de tràfic general.
- **backend:** defineix on s'enviarà el tràfic. En aquest cas, s'enviarà al servei Apache que s'està definint a Kubernetes amb el nom generat per `{{ include "lamp.fullname" $ }}`. El servei serà accessible al port **8080** (com s'ha especificat anteriorment en el servei de tipus **NodePort**).

Creació del fitxer `storage.yaml`

Aquest fitxer `storage` gestiona els PV (**PersistentVolume**) i PVC (**PersistentVolumeClaim**) que son els objectes de Kubernetes que serveixen per a gestionar i garantir l'emmagatzematge persistent per a aplicacions dins del clúster.

```
# PersistentVolume for MySQL
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: {{ .Release.Name }}-mysql-pv
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: mysql
spec:
  capacity:
    storage: 10Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: {{ .Values.persistentVolume.storageClass }}
  hostPath:
    path: /vagrant/data/mysql
---

# PersistentVolumeClaim for MySQL
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: {{ .Release.Name }}-mysql-pvc
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: mysql
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: {{ .Values.persistentVolume.storageClass }}
```

```

resources:
  requests:
    storage: 10Gi

# PersistentVolume for PHP
---
apiVersion: v1
kind: PersistentVolume
metadata:
  name: {{ .Release.Name }}-php-pv
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: php
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: {{ .Values.persistentVolume.storageClass }}
  hostPath:
    path: /vagrant/data/php
---

# PersistentVolumeClaim for PHP
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: {{ .Release.Name }}-php-pvc
  namespace: {{ .Values.namespace }}
  labels:
    app: {{ .Chart.Name }}
    component: php
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: {{ .Values.persistentVolume.storageClass }}
  resources:
    requests:
      storage: 5Gi

```

PersistentVolumeClaim.storage: Assegura l'espai d'emmagatzematge persistent necessari per a guardar les dades de la base de dades MySQL i fitxers d'aplicacions o configuracions específiques necessàries per al servei PHP.

PVC Garantitza que, fins i tot si el pod es recrea, les dades emmagatzemades es mantindran disponibles. A més, l'accés definit com a **ReadWriteOnce**, el que significa que

només un node pot muntar aquest volum ahora.

Creació del fitxer values.yaml

El fitxer `values.yaml` és un component clau en Helm. Proporciona els valors predeterminats per a les variables que s'utilitzen en les plantilles de Kubernetes. Aquest fitxer serveix com a lloc central per emmagatzemar configuracions específiques que poden ser reutilitzades i sobreescrites durant el procés d'instal·lació o actualització d'un Chart.

```
namespace: "lamp"

persistentVolume:
  storageClass: "default"

apache:
  image: httpd:2.4
  port: 80
  targetPort: 80
  nodePort: 30080

mysql:
  image: mysql:5.7
  port: 3306
  targetPort: 3306
  rootPassword: "U3VwM3JTM2NyM3RSMDUUDQzNDQwNXcwckQh"
  database: d2lraWRi
  user: d2lraXVzZXI=
  password: "VzFrMVA0MzQ0MDV3MHJkLg=="
  mycnfSettings:
    innodb_use_native_aio: "0"

php:
  image: php:8.1-fpm
  port: 9000
  targetPort: 9000
  memoryLimit: "256M"
  maxExecutionTime: 120
  uploadMaxFileSize: "20M"
  postMaxSize: "25M"
  extensions:
    - gd
    - mbstring
    - xml
    - intl
    - json
    - curl
```

```

- zip
- opcache
- fileinfo
iniSettings:
  error_reporting: "E_ALL & ~E_NOTICE & ~E_DEPRECATED"
  display_errors: "Off"
  log_errors: "On"
  opcache.enable: "1"
  session.save_handler: "files"
  session.save_path: "/tmp"

autoscaling:
  enabled: true
  minReplicas: 1
  maxReplicas: 5
  targetCPUUtilizationPercentage: 80
  targetMemoryUtilizationPercentage: 80

ingress:
  enabled: true
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
  hosts:
    - host: "localhost"
      paths:
        - path: "/"
          pathType: Prefix

```

Values globals

Conté les configuracions globals per a l'aplicació `lamp`:

- `namespace: "lamp"` defineix el nom del namespace on desplegar l'aplicació.
- `persistentVolume.storageClass: "default"` En fer proves locals, com Minikube o Docker Desktop, configurarem una `StorageClass` com `default` o alguna altra que s'ajusti al nostre entorn.

Values per apache

Conté les configuracions per a l'aplicació Apache:

- **image:** Defineix la imatge del contenidor Docker que s'utilitza per Apache (en aquest cas, `httpd:2.4`).
- **port:** El port al qual Apache estarà disponible dins del contenidor (en aquest cas, `80`).

Values per mysql

Configura el servei de base de dades MySQL:

- **image:** La imatge de MySQL que s'utilitza per crear el contenidor.
- **port:** El port al qual MySQL estarà disponible dins del contenidor (en aquest cas, `3306`).
- **rootPassword:** Consta de la contrasenya de l'usuari `root` de MySQL.
- **database:** El nom de la base de dades inicial que es crearà quan MySQL s'iniciï.
- **user:** L'usuari de la base de dades que serà creat.
- **password:** La contrasenya associada a l'usuari de la base de dades.

Values per php

Defineix la configuració de PHP:

- **image:** Defineix quina imatge de PHP utilitzar al contenidor.
- **port:** El port al qual PHP estarà disponible dins del contenidor (en aquest cas, `9000`).
- **memoryLimit:** Controla la quantitat de memòria que una instància de PHP pot utilitzar.
- **maxExecutionTime:** Defineix el temps màxim que pot executar-se un script PHP abans de ser interromput.
- **uploadMaxFileSize:** Estableix la mida màxima dels fitxers que es poden pujar mitjançant formularis.
- **extensions:** Permet definir una llista d'extensions de PHP necessàries per a l'aplicació.
 - **gd:** Per a la manipulació d'imatges.
 - **mbstring:** Necessària per gestionar cadenes multibyte.
 - **xml i json:** Per processar dades estructurades.
 - **intl:** Per funcionalitats de localització i internacionalització.
 - **opcache:** Per millorar el rendiment.
 - **fileinfo:** Per determinar tipus `MIME` de fitxers pujats.
 - **curl:** Per consultes HTTP.
- **iniSettings:**
Configuracions personalitzades del fitxer `php.ini`, com la gestió d'errors i límits de mida de publicació.

Values per autoscaling

Defineix les opcions d'autoscaling (màxim i mínim nombre de rèpliques i percentatges d'utilització de CPU i memòria).

- **enabled:** Activa l'autoscaling amb el valor a **true**
- **minReplicas:** Nombre mínim de rèpliques, en el nostre cas **1**
- **maxReplicas:** Nombre màxim de rèpliques, en el nostre cas **5**
- **targetCPUUtilizationPercentage:** Percentatge d'ús de CPU per a l'autoscaling, **80%** al nostre cas
- **targetMemoryUtilizationPercentage:** Percentatge d'ús de memòria per a l'autoscaling, també **80%**.

Values per ingress

- Amb la configuració de l'Ingress configurat (valor a **true**), es pot accedir al teu sistema LAMP mitjançant un **NodePort** configurat al fitxer Service, al nostre cas serà **localhost:8080**. El tràfic serà redirigit des del port **8080** del ordinador amfitrió a través de Kubernetes fins al servei Apache que s'exposa a aquest port.

Aquest fitxer proporciona la flexibilitat per personalitzar fàcilment les variables quan es desplega l'aplicació amb Helm. Es poden sobreescriure els valors predeterminats mitjançant l'ús de l'argument **--set** durant la instal·lació o actualització del Chart, o també mitjançant un fitxer de configuració personalitzat.

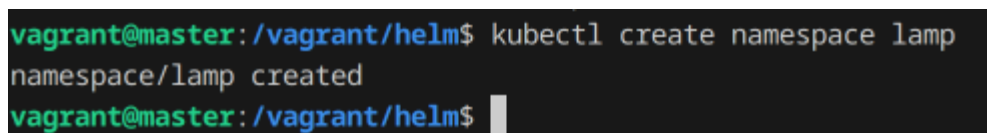
Per desplegar l'aplicació en un **namespace** específic utilitzant **Helm**, seguim els següents passos:

Crear un namespace

Si el namespace que es vol utilitzar encara no existeix, el primer pas serà crear-lo. Per fer-ho executarem el següent comandament:

```
kubectl create namespace nom-del-namespace
```

On **nom-del-namespace** és el nom del namespace on es vol desplegar l'aplicació, per exemple **lamp**.



```
vagrant@master:/vagrant/helm$ kubectl create namespace lamp
namespace/lamp created
vagrant@master:/vagrant/helm$
```

Desplegar l'aplicació amb Helm i crear un namespace específic

Quan es desplega l'aplicació amb Helm, podem especificar el namespace mitjançant

l'argument `--namespace`, si a més afegim `--create-namespace` el crearem.

Per exemple, si es vol desplegar l'aplicació en un namespace anomenat `lamp`, el comandament seria:

```
helm install lamp ./lamp --create-namespace --namespace lamp
```

Aquest comandament desplegarà l'aplicació que tenim a la carpeta `./lamp` dins del namespace `lamp`.

Comprovar el desplegament en el namespace

Després de desplegar l'aplicació, podem comprovar que els recursos han estat creats en el namespace específic mitjançant:

```
kubectl get all -o wide --namespace lamp
```

Aquest comandament mostrarà tots els recursos (com els Pods, services, deployments, etc.) que s'han creat dins del namespace `lamp`.

```
vagrant@master:~/vagrant/helm$ kubectl get all -o wide --namespace lamp
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/lamp-apache-6b6bd5b45b-129dp	1/1	Running	0	3m17s	10.244.2.8	worker2	<none>	<none>
pod/lamp-mysql-67864c8d9c-5n888	0/1	Pending	0	3m17s	<none>	<none>	<none>	<none>
pod/lamp-php-7dfbdb966-4wxxw	1/1	Running	0	3m17s	10.244.1.9	worker1	<none>	<none>

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
service/lamp-apache-service	NodePort	10.104.187.149	<none>	80:30080/TCP	3m17s	app=lamp,component=apache
service/lamp-mysql-service	ClusterIP	10.110.5.65	<none>	3306/TCP	3m17s	app=lamp,component=mysql
service/lamp-php-service	ClusterIP	10.107.10.109	<none>	9000/TCP	3m17s	app=lamp,component=php

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	CONTAINERS	IMAGES	SELECTOR
deployment.apps/lamp-apache	1/1	1	1	3m17s	apache	httpd:2.4	app=lamp,component=apache
deployment.apps/lamp-mysql	0/1	1	0	3m17s	mysql	mysql:5.7	app=lamp,component=mysql
deployment.apps/lamp-php	1/1	1	1	3m17s	php	php:8.1-fpm	app=lamp,component=php

NAME	DESIRED	CURRENT	READY	AGE	CONTAINERS	IMAGES	SELECTOR
replicaset.apps/lamp-apache-6b6bd5b45b	1	1	1	3m17s	apache	httpd:2.4	app=lamp,component=apache,pod-template-hash=6b6bd5b45b
replicaset.apps/lamp-mysql-67864c8d9c	1	1	0	3m17s	mysql	mysql:5.7	app=lamp,component=mysql,pod-template-hash=67864c8d9c
replicaset.apps/lamp-php-7dfbdb966	1	1	1	3m17s	php	php:8.1-fpm	app=lamp,component=php,pod-template-hash=7dfbdb966

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/lamp-apache	Deployment/lamp-apache	<unknown>/80%	1	5	1	3m17s
horizontalpodautoscaler.autoscaling/lamp-mysql	Deployment/lamp-mysql	<unknown>/80%	1	5	1	3m17s
horizontalpodautoscaler.autoscaling/lamp-php	Deployment/lamp-php	<unknown>/80%	1	5	1	3m17s

```
vagrant@master:~/vagrant/helm$
```

Afegir el namespace al fitxer `values.yaml` i als templates

Una altra opció (com hem fet) és especificar el namespace directament al fitxer `values.yaml` i a les templates de Helm, molt útil si tenim més d'una aplicació i namespace, a més també ens serà útil per controlar el desplegament desde ArgoCD més endavant.

Per fer-ho així, afegirem una línia al fitxer `values.yaml` per especificar el namespace en què volem que s'instal·li l'aplicació:

```
namespace: "lamp"
```

I a les templates seria al bloc metadata de cada component, per exemple al fitxer `deployment.yaml`

```

apiVersion: apps/v1
kind: Deployment
metadata:
name: {{ .Release.Name }}-apache
labels:
app: {{ .Chart.Name }}
component: apache
namespace: {{ .Release.Namespace }}
[...]
```

Després, en executar el comandament de Helm, només caldrà executar:

```
helm install lamp ./lamp --create-namespace --namespace lamp
```

1.5. Instal·lació de Kubernetes Dashboard

El Kubernetes Dashboard és una interfície d'usuari web que permet gestionar i solucionar problemes d'aplicacions dins del clúster de Kubernetes. S'hi pot veure l'estat dels recursos, els registres i gestionar els serveis i les implantacions. Es pot desplegar amb Helm o kubectl i l'accés es fa a través d'un proxy segur amb kubectl. Per més informació detallada sobre la instal·lació i configuració, es pot consultar la documentació oficial de Kubernetes: <https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/>.

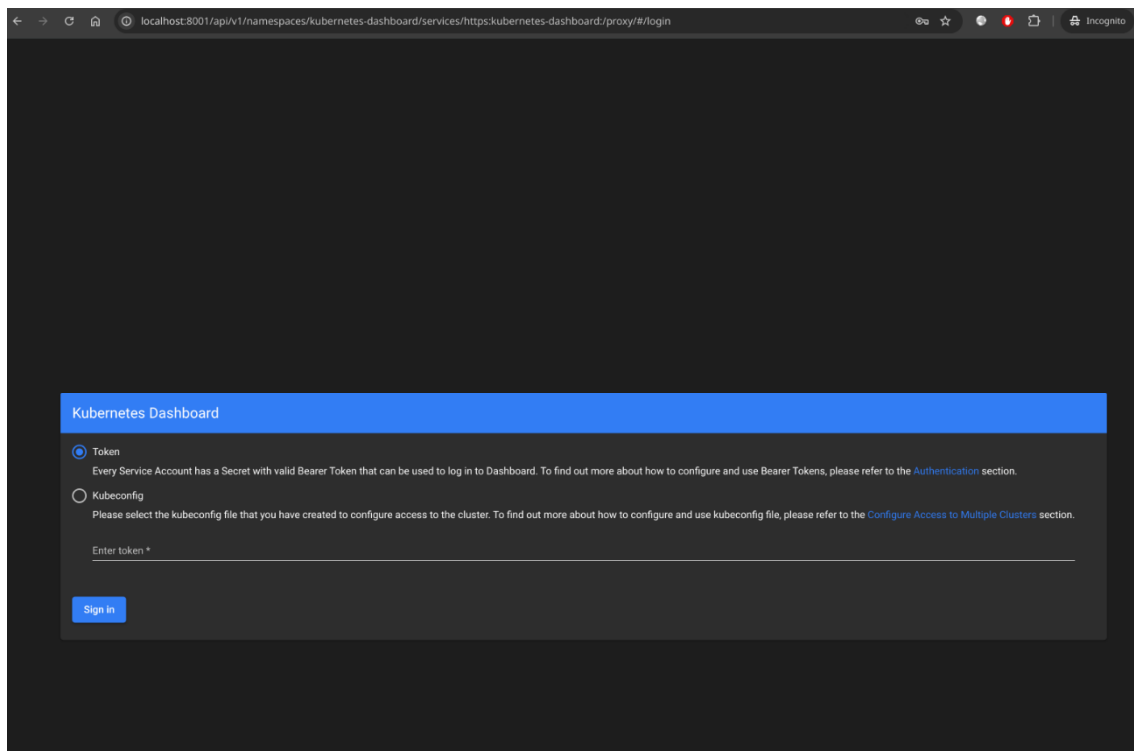
Com la nostra versió de Kubernetes amb Minikube no permet fer servir Helm (només es possible a partir de la 1.21 i tenim la 1.20.15), instal·larem el dashboard amb kubectl amb la darrera versió disponible del repositori <https://github.com/kubernetes/dashboard/releases/tag/v2.4.0>.

Executarem al node master que ens instal·larà automàticament tots els components necessaris per desplegar el nostre Dashboard.

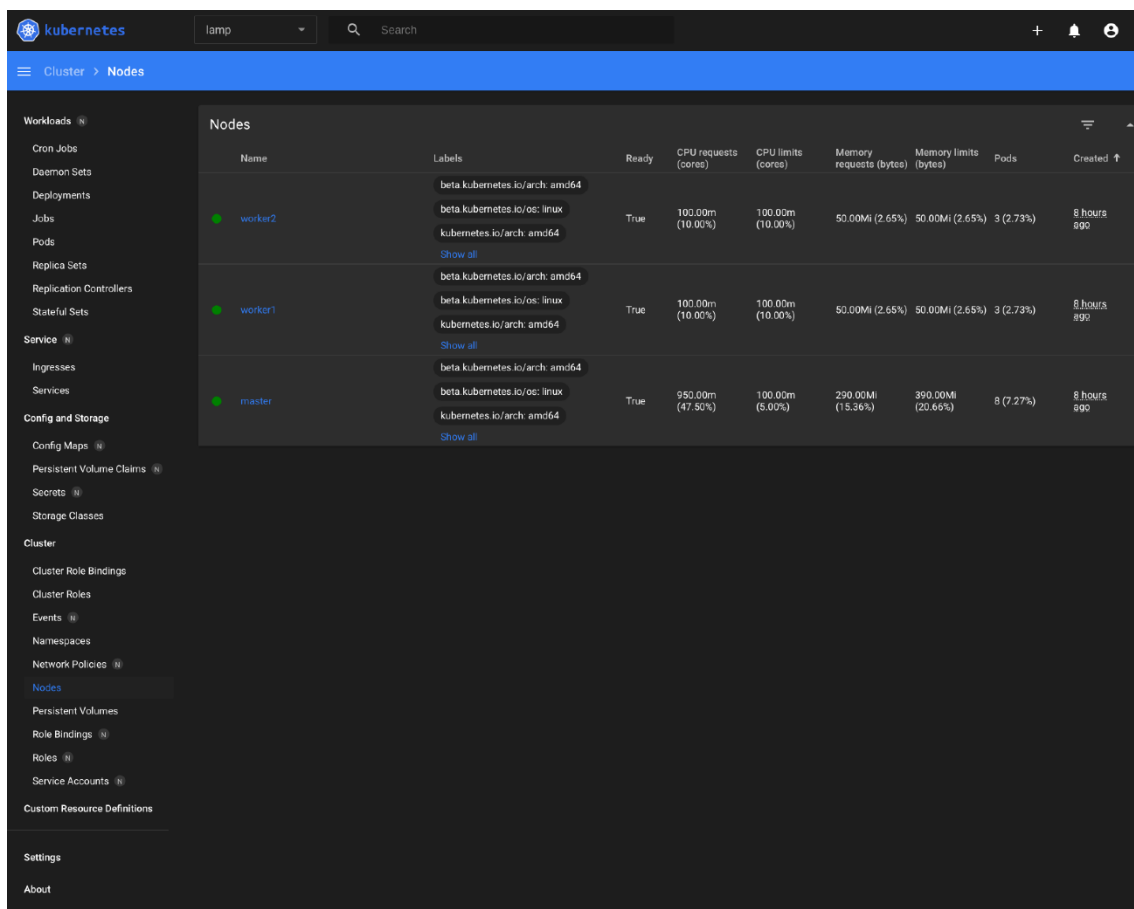
```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.4.0/aio/deploy/recommended.yaml
```

```

vagrant@master:~$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.4.0/aio/deploy/recommended.yaml
namespace/kubernetes-dashboard created
serviceaccount/kubernetes-dashboard created
service/kubernetes-dashboard created
secret/kubernetes-dashboard-certs created
secret/kubernetes-dashboard-csrf created
secret/kubernetes-dashboard-key-holder created
configmap/kubernetes-dashboard-settings created
role.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrole.rbac.authorization.k8s.io/kubernetes-dashboard created
rolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
clusterrolebinding.rbac.authorization.k8s.io/kubernetes-dashboard created
deployment.apps/kubernetes-dashboard created
service/dashboard-metrics-scraper created
deployment.apps/dashboard-metrics-scraper created
```

Un cop dins ja podem navegar pels objectes del nostre clúster



1.6. Migració de MediaWiki de l'entorn monolític a microserveis

Migrar MediaWiki de l'entorn monolític a microserveis amb HELM implica separar les funcionalitats de l'aplicació en serveis independents com la base de dades, el servidor web, i altres components.

- Analitzar arquitectura actual: Identificar els components del sistema MediaWiki.
- Crear Helm Chart: Configurar desplegaments per cada servei (MediaWiki, base de dades).
- Utilitzar ConfigMaps/Secrets: Per configuracions com LocalSettings.php.
- Ingress i serveis: Configurar ingress per exposar l'aplicació.
- Persistència: Definir PersistentVolumes per fitxers estàtics i dades.

Partirem de la base que hem creat abans amb els Charts de LAMP per crear la nostra aplicació per la MediaWiki.

Afegir configuració personalitzada a Apache

Inclouem un VirtualHost al fitxer `httpd.conf` del contenidor Apache:

- Modificarem el ConfigMap perquè inclogui el fitxer modificat `/usr/local/apache2/conf/httpd.conf`

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-apache-config
  namespace: {{ .Values.namespace }}
data:
  httpd.conf: |
    ServerName localhost
    ServerRoot "/usr/local/apache2"
    Listen 80
    [...]
    <VirtualHost *:80>
      ServerName localhost
      ProxyPass / php://{{ .Release.Name }}-php-
service.{{ .Values.namespace }}:{{ .Values.php.port }}/
      ProxyPassReverse / php://{{ .Release.Name }}-php-
service.{{ .Values.namespace }}:{{ .Values.php.port }}/
      <FilesMatch "\.php$">
        SetHandler "proxy:unix:/path/to/php-fpm.sock|fcgi://localhost"
      </FilesMatch>
    </VirtualHost>
    [...]
```

També hem de modificar el `deployment` per incloure un volum pel `ConfigMap`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-apache
  labels:
    app: {{ .Chart.Name }}
    component: apache
    namespace: {{ .Release.namespace }}
spec:
  containers:
  [...]

  volumeMounts:
  - name: apache-config
    mountPath: /usr/local/apache2/conf/httpd.conf
    subPath: httpd.conf
  volumes:
  - name: apache-config
    configMap:
      name: {{ .Release.Name }}-apache-config
```

Incloure LocalSettings.php

Afegirem el fitxer `LocalSettings.php` de l'entorn monolític al contenidor PHP/Apache, per això utilitzarem un `ConfigMap` per injectar aquest fitxer al camí que pertorqui del nostre `LocalSettings.php` al nostre cas serà `/var/www/html`

```
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-mediawiki-settings
  namespace: {{ .Release.namespace }}
data:
  LocalSettings.php: |
  [...]
  # Contingut del teu fitxer LocalSettings.php
  [...]
```

Modificarem el `deployment` de apache per incloure el nou `ConfigMap`

```
apiVersion: apps/v1
kind: Deployment
```

```

metadata:
  name: {{ .Release.Name }}-apache
  labels:
    app: {{ .Chart.Name }}
    component: apache
  namespace: {{ .Release.namespace }}
spec:
  [...]
  volumeMounts:
  - name: php-data
    mountPath: /var/www/html
  - name: mediawiki-settings
    mountPath: /var/www/html/LocalSettings.php
    subPath: LocalSettings.php

```

Per fer que els valors de `LocalSettings.php` de MediaWiki agafin els secrets de MySQL, podem fer el següent:

Modificar el `ConfigMap` per utilitzar variables d'entorn:

```

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ .Release.Name }}-mediawiki-settings
  namespace: {{ .Values.namespace }}
data:
  LocalSettings.php: |
    [...]
    $wgDBserver = getenv('DB_SERVER');
    $wgDBname = getenv('MYSQL_DATABASE');
    $wgDBuser = getenv('MYSQL_USER');
    $wgDBpassword = getenv('MYSQL_PASSWORD');
    [...]

```

Modificar el `Deployment` per injectar els `Secrets` i els `ConfigMap` nous:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-apache
  labels:
    app: {{ .Chart.Name }}
    component: apache
  [...]
spec:
  containers:
  [...]
  env:

```

```

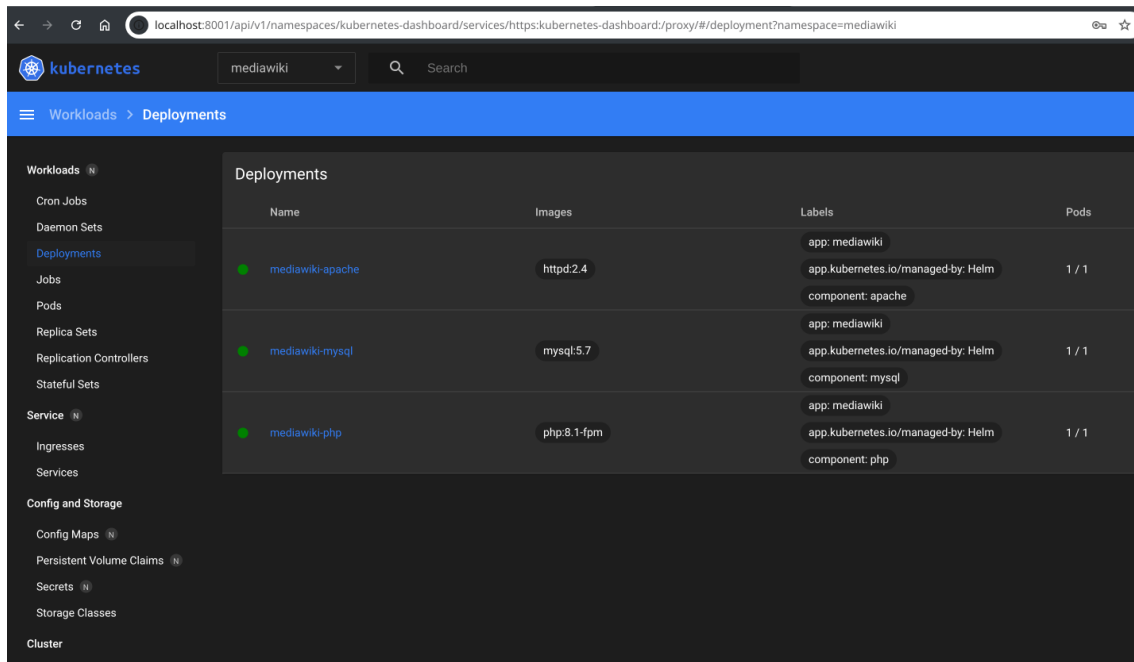
- name: PHP_MEMORY_LIMIT
  value: "{{ .Values.php.memoryLimit }}"
- name: PHP_MAX_EXECUTION_TIME
  value: "{{ .Values.php.maxExecutionTime }}"
- name: DB_SERVER
  value: "{{ .Release.Name }}-mysql-service.{{ .Values.namespace }}"
- name: MYSQL_DATABASE
  valueFrom:
    secretKeyRef:
      name: {{ .Release.Name }}-mysql-secrets
      key: mysql-database
- name: MYSQL_USER
  valueFrom:
    secretKeyRef:
      name: {{ .Release.Name }}-mysql-secrets
      key: mysql-user
- name: MYSQL_PASSWORD
  valueFrom:
    secretKeyRef:
      name: {{ .Release.Name }}-mysql-secrets
      key: mysql-password
volumeMounts:
- name: php-data
  mountPath: /var/www/html
- name: mediawiki-settings
  mountPath: /var/www/html/LocalSettings.php
  subPath: LocalSettings.php
volumes:
- name: php-data
  persistentVolumeClaim:
    claimName: {{ .Release.Name }}-php-pvc
- name: mediawiki-settings
  configMap:
    name: {{ .Release.Name }}-mediawiki-settings

```

Així els secrets de MySQL es passen com a variables d'entorn dins del contenidor.

Finalment instal·lem la nostra aplicació

```
helm install mediawiki ./mediawiki --create-namespace --namespace mediawiki
```



Ara ja la tenim desplegada però encara no té totes les dades de la MediaWiki que hi havia a l'entorn monolític, hem de acabar la migració de les dades del PHP i del MySQL que després persistiran als PVC.

Migració de la MediaWiki

Seguint el procediment de

https://www.mediawiki.org/wiki/Manual:Running_MediaWiki_on_Debian_or_Ubuntu#Get_latest_MediaWiki

Instal·lem la MediaWiki al pod del PHP:

```
cd /tmp/  
wget https://releases.wikimedia.org/mediawiki/1.43/mediawiki-1.43.0.tar.gz  
tar -xvzf /tmp/mediawiki-*.tar.gz  
mv mediawiki-*/ * /var/www/html/mediawiki
```

Amb aquesta part tindrem el contingut bàsic de la MediaWiki, si tenim més dades haurem de fer un backup de `/var/lib/mediawiki` del servidor anterior i recuperar-les.

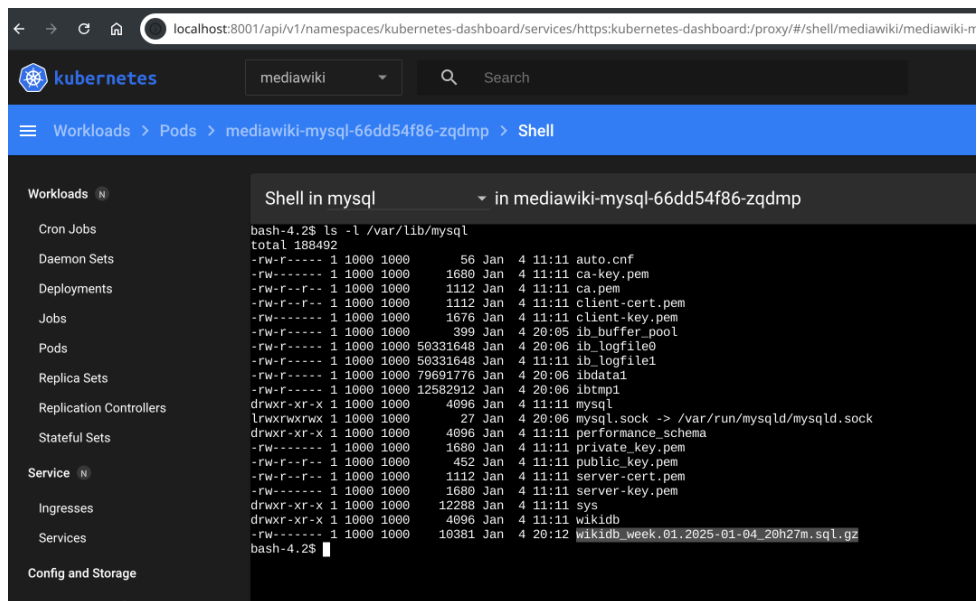
Continuem ara amb el procediment de recuperació de la DB seguint les passes que tenim a https://www.mediawiki.org/wiki/Manual:Backing_up_a_wiki. Farem un backup del servidor antic per recuperar la nostra DB de MediaWiki en Kubernetes.

Tenim automysqlbackup instal·lat al servidor antic i hem fet els següents backups:

```
# apt show automysqlbackup
Package: automysqlbackup
Version: 2.6+debian.4-4
Priority: extra
Section: universe/admin
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Thomas Goirand <zigo@debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 58.4 kB
Depends: bsd-mailx | mailx, default-mysql-client | virtual-mysql-client
Recommends: mutt
Homepage: http://sourceforge.net/projects/automysqlbackup/
Download-Size: 14.6 kB
APT-Manual-Installed: yes
APT-Sources: http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages
Description: daily, weekly and monthly backup for your MySQL database

# find /var/lib/automysqlbackup
/var/lib/automysqlbackup
/var/lib/automysqlbackup/daily
/var/lib/automysqlbackup/daily/sys
/var/lib/automysqlbackup/daily/wikidb
/var/lib/automysqlbackup/monthly
/var/lib/automysqlbackup/weekly
/var/lib/automysqlbackup/weekly/sys
/var/lib/automysqlbackup/weekly/sys/sys_week.01.2025-01-04_20h27m.sql.gz
/var/lib/automysqlbackup/weekly/wikidb
/var/lib/automysqlbackup/weekly/wikidb/wikidb_week.01.2025-01-04_20h27m.sql.gz
#
```

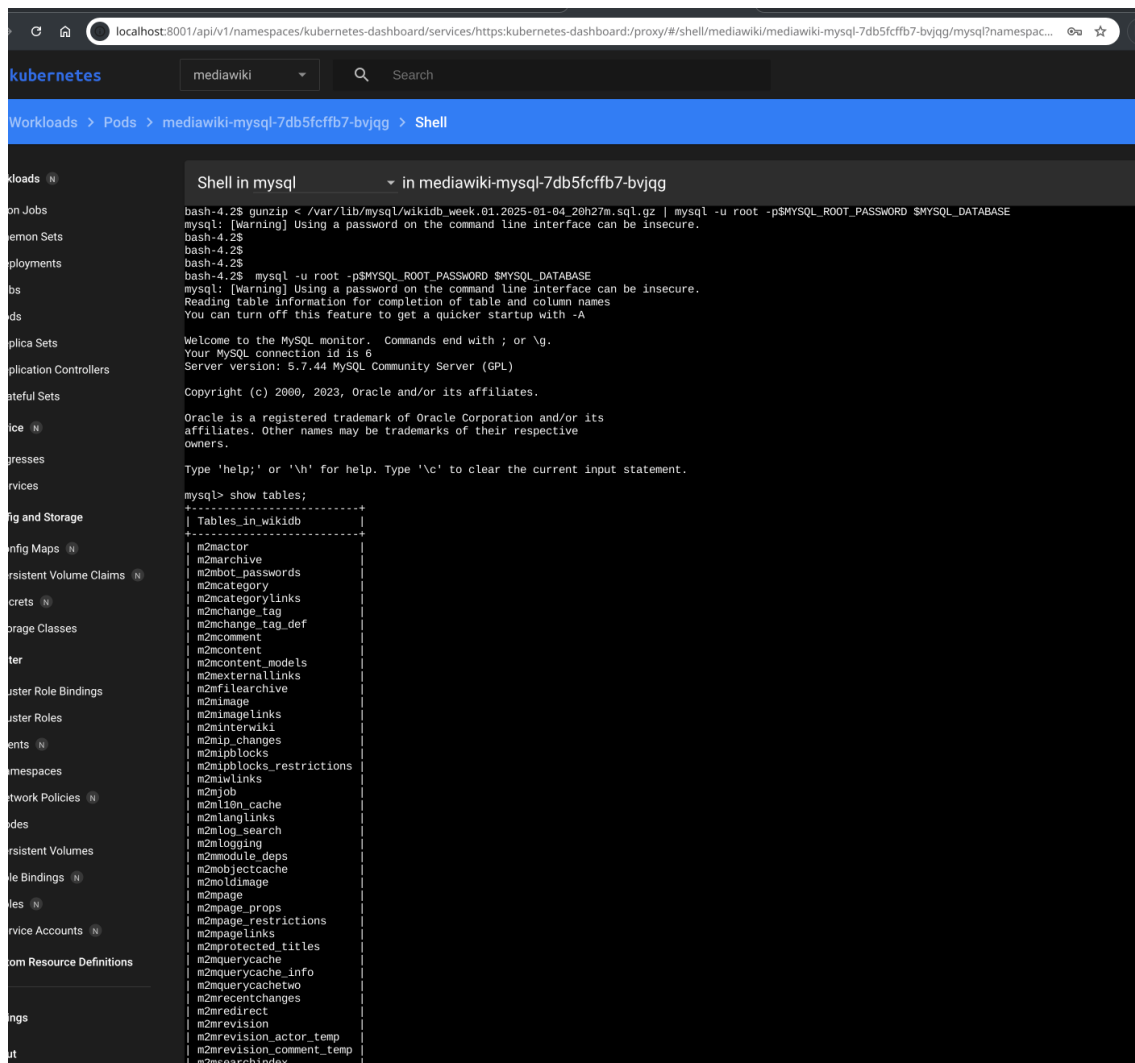
Com Vagrant permet compartició de fitxers amb l'amfitrió a través de NFS amb el directori vagrant, només cal que copiem el fitxer "weekly" en vagrant/data/mysql que quedarà mapejat al pod en /var/lib/mysql segons el nostre PVC configurat.



Ens connectem al pod de MySQL, creem la nostra DB recuperant-la del fitxer:

```
gunzip < /var/lib/mysql/wikidb_week.01.2025-01-04_20h27m.sql.gz |mysql -u
root -p $MYSQL_ROOT_PASSWORD $MYSQL_DATABASE
```

Comprovem que estiguin les dades connectant-nos al MySQL i executant una comanda com ara `show tables`



```
bash-4.2$ gunzip < /var/lib/mysql/wikidb_week.01.2025-01-04_20h27m.sql.gz | mysql -u root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE
mysql: [warning] Using a password on the command line interface can be insecure.
bash-4.2$
bash-4.2$ mysql -u root -p$MYSQL_ROOT_PASSWORD $MYSQL_DATABASE
mysql: [warning] Using a password on the command line interface can be insecure.
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.7.44 MySQL Community Server (GPL)

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show tables;
+-----+
| Tables_in_wikidb |
+-----+
| m2mactor          |
| m2marchive        |
| m2mbot_passwords  |
| m2mcategory       |
| m2mcategorylinks   |
| m2mchange_tag     |
| m2mchange_tag_def |
| m2mcomment        |
| m2mcontent        |
| m2mcontent_models |
| m2mexternallinks  |
| m2mfilearchive    |
| m2mimage          |
| m2mimagelinks     |
| m2minterwiki      |
| m2mip_changes     |
| m2mipblocks       |
| m2mipblocks_restrictions |
| m2miwlinks        |
| m2mjob            |
| m2mliön_cache     |
| m2mlanglinks      |
| m2mlog_search     |
| m2mlogging        |
| m2mmodule_deps    |
| m2mobjectcache    |
| m2moidimage       |
| m2mpage           |
| m2mpage_props     |
| m2mpage_restrictions |
| m2mpagelinks      |
| m2mprotected_titles |
| m2mquerycache     |
| m2mquerycache_info |
| m2mquerycachetwo  |
| m2mrecentchanges  |
| m2mredirect       |
| m2mrevision       |
| m2mrevision_actor_temp |
| m2mrevision_comment_temp |
| m2msearchindex    |
```

Amb tot ja hauríem de ser capaços d'arrencar la nostra aplicació tal com estava al servidor monolític, reiniciem el deployment de HELM amb:

```
helm upgrade --install mediawiki ./mediawiki --namespace mediawiki
```

i comprovem que la nostra wiki esta corrent al Kubernetes:

